

# 北京理工大学

本科生毕业设计（论文）

## LeRobot 平台感知—推理—执行流程的 系统级性能剖析研究

System-Level Performance Profiling of  
Perception-Inference-Execution Pipeline  
in the LeRobot Framework

学 院： 计算机学院

专 业： 软件工程

班 级： 08012204

学 生 姓 名： 俞乐楠

学 号： 1120221303

指 导 教 师： 王勇

2026 年 4 月 20 日

## 原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在指导老师的指导下独立进行研究所取得的成果。除文中已经注明引用的内容外，本文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

特此申明。

本人签名：\_\_\_\_\_ 日期：\_\_\_\_\_ 年 \_\_\_\_ 月 \_\_\_\_ 日

## 关于使用授权的声明

本人完全了解北京理工大学有关保管、使用毕业设计（论文）的规定，其中包括：学校有权保管、并向有关部门送交本毕业设计（论文）的原件与复印件；学校可以采用影印、缩印或其它复制手段复制并保存本毕业设计（论文）；学校可允许本毕业设计（论文）被查阅或借阅；学校可以学术交流为目的，复制赠送和交换本毕业设计（论文）；学校可以公布本毕业设计（论文）的全部或部分内容。

本人签名：\_\_\_\_\_ 日期：\_\_\_\_\_ 年 \_\_\_\_ 月 \_\_\_\_ 日

指导教师签名：\_\_\_\_\_ 日期：\_\_\_\_\_ 年 \_\_\_\_ 月 \_\_\_\_ 日

# LeRobot 平台感知—推理—执行流程的 系统级性能剖析研究

## 摘 要

[TODO: 正式摘要待实验完成后撰写, 约 300-500 字。]

本文以树莓派 5 (ARM Cortex-A76, 无 GPU) 为目标平台, 对 LeRobot 框架下 ACT 机器人学习策略的感知—推理—执行三阶段时序特征进行系统级剖析。通过在 Python 应用层、PyTorch 运行时层、Linux 内核层及硬件 I/O 层的多工具联合测量 (Python logging / torch.profiler / ftrace / strace), 端到端量化了各阶段的时间开销, 定位了主要性能瓶颈, 并从算法、运行时、系统架构三个维度梳理了可行的优化路径。

实验结果表明, ACT 单次推理耗时 2000–4000 ms, 占总循环时间的 99% 以上, 是系统主瓶颈; 观测和执行阶段开销极低 (各约 5 ms), 系统调用本身的开销可忽略不计, 排除了 I/O 路径为主要瓶颈的假设。在 chunk\_size=100 配置下, 系统实际帧率约 18–19 fps, 距 30 fps 目标仍有明显差距, 瓶颈根源在于 ARM CPU 矩阵乘法 (GEMM) 计算能力不足, 而非感知复杂度或任务类型。

**关键词:** LeRobot; ACT; 性能剖析; ARM 推理; chunk\_size; 感知-推理-执行

# System-Level Performance Profiling of Perception-Inference-Execution Pipeline in the LeRobot Framework

## Abstract

[TODO: English abstract to be finalized after experiments complete, approximately 300 words.]

This paper presents a system-level performance analysis of the ACT (Action Chunking with Transformers) policy running on a Raspberry Pi 5 (ARM Cortex-A76, no GPU) within the LeRobot framework. By combining multi-layer profiling tools across the Python application layer, PyTorch runtime layer, Linux kernel layer, and hardware I/O layer (Python logging, torch.profiler, ftrace, strace), we quantify the end-to-end time distribution across perception, inference, and execution stages, identify the primary bottleneck, and systematically evaluate feasible optimization paths.

Results show that a single ACT inference takes 2000–4000 ms, accounting for over 99% of the total loop time, making it the dominant bottleneck. Perception and execution stages contribute only  $\approx 5$  ms each; system call overhead is negligible, ruling out I/O as a bottleneck. At `chunk_size=100`, the system achieves  $\approx 18$ –19 fps against a 30 fps target. The root cause is insufficient ARM CPU matrix-multiplication (GEMM) throughput, independent of task visual complexity or object count.

**Key Words:** LeRobot; ACT; performance profiling; ARM inference; `chunk_size`; perception-inference-execution

## 目录

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>第一章 引言</b>                                            | <b>1</b>  |
| 1.1 研究背景与意义 . . . . .                                    | 1         |
| 1.2 国内外研究现状 . . . . .                                    | 2         |
| 1.3 研究目标 . . . . .                                       | 3         |
| <b>第二章 实验环境构建</b>                                        | <b>4</b>  |
| 2.1 实验平台选型 . . . . .                                     | 4         |
| 2.2 推理循环与软件栈 . . . . .                                   | 6         |
| 2.3 三类典型工作负载特征 . . . . .                                 | 7         |
| <b>第三章 性能分析</b>                                          | <b>9</b>  |
| 3.1 工具链选型 . . . . .                                      | 9         |
| 3.2 感知阶段性能剖析 . . . . .                                   | 14        |
| 3.2.1 摄像头异步读取机制 . . . . .                                | 14        |
| 3.2.2 是否进入内核 . . . . .                                   | 15        |
| 3.2.3 OpenCV 到内核的边界 . . . . .                            | 16        |
| 3.2.4 时间剖析 . . . . .                                     | 17        |
| 3.3 推理阶段性能剖析 . . . . .                                   | 17        |
| 3.3.1 <code>select_action</code> 与动作队列 . . . . .         | 17        |
| 3.3.2 一次完整 ACT 推理的数据流 . . . . .                          | 18        |
| 3.3.3 时间剖析: <code>chunk_size</code> 参数扫描 . . . . .       | 19        |
| 3.3.4 单次 ACT 推理耗时分解 . . . . .                            | 21        |
| 3.3.5 PyTorch 线程行为与推理优化研究现状 . . . . .                    | 22        |
| 3.4 执行阶段性能剖析 . . . . .                                   | 22        |
| 3.4.1 LeRobot 执行路径软件栈 . . . . .                          | 22        |
| 3.4.2 <code>sync_write</code> 写链路: 从电机总线到串口写入 . . . . .  | 22        |
| 3.4.3 <code>sync_read</code> 读链路: 从请求帧到逐电机轮询回包 . . . . . | 23        |
| 3.4.4 关键性能结论 . . . . .                                   | 24        |
| <b>第四章 优化</b>                                            | <b>25</b> |
| 4.1 优化方向的来源: 从瓶颈结构反推 . . . . .                           | 25        |

|       |                                                  |    |
|-------|--------------------------------------------------|----|
| 4.1.1 | 第三章瓶颈结论回顾 . . . . .                              | 25 |
| 4.1.2 | 同步流水线下的 CPU 时间浪费 . . . . .                       | 25 |
| 4.1.3 | 三个候选异步点 . . . . .                                | 26 |
| 4.2   | 舵机执行通路异步化的可行性分析 . . . . .                        | 26 |
| 4.2.1 | 半双工 TTL 总线的物理约束 . . . . .                        | 26 |
| 4.2.2 | 异步写为什么不可行 . . . . .                              | 26 |
| 4.2.3 | 小结 . . . . .                                     | 27 |
| 4.3   | 舵机观测通路异步化的可行性分析 . . . . .                        | 28 |
| 4.3.1 | 上游 TODO: 基于 txPacket/rxPacket 拆分的流水线方案 . . .     | 28 |
| 4.3.2 | 与相机后台线程式异步的本质区别 . . . . .                        | 28 |
| 4.3.3 | 落地代价: 驱动改造与观测滞后 . . . . .                        | 29 |
| 4.3.4 | 收益估算 . . . . .                                   | 29 |
| 4.3.5 | 小结 . . . . .                                     | 30 |
| 4.4   | 推理异步化: 本章重点 . . . . .                            | 30 |
| 4.4.1 | 设计动机: 让推理与执行真正并行 . . . . .                       | 30 |
| 4.4.2 | LeRobot 官方双进程架构 . . . . .                        | 30 |
| 4.4.3 | 三个核心机制 . . . . .                                 | 31 |
| 4.4.4 | 可行性不等式 $N \geq 2T_{\text{infer}}F$ 的推导 . . . . . | 31 |
| 4.4.5 | 不等式在本平台的代入 . . . . .                             | 32 |
| 4.4.6 | 树莓派 5 单机部署的 CPU 争抢 . . . . .                     | 33 |
| 4.5   | 异步推理收益的实验验证 . . . . .                            | 33 |
| 4.5.1 | 实验目的与论证逻辑 . . . . .                              | 33 |
| 4.5.2 | 实验设计 . . . . .                                   | 33 |
| 4.5.3 | 控制变量 . . . . .                                   | 34 |
| 4.5.4 | 测量指标 . . . . .                                   | 34 |
| 4.5.5 | 实验流程 . . . . .                                   | 35 |
| 4.5.6 | 预期结果与待验证假设 . . . . .                             | 36 |
| 4.5.7 | 实验结果呈现 (待回填) . . . . .                           | 36 |
| 4.5.8 | 讨论: 实验结论的工程含义 . . . . .                          | 37 |
| 4.6   | 优化路线图与本章小结 . . . . .                             | 37 |
| 4.6.1 | 优化方向矩阵 . . . . .                                 | 37 |
| 4.6.2 | 三阶段优化路线 . . . . .                                | 38 |

|                         |           |
|-------------------------|-----------|
| 4.6.3 与第五章的衔接 . . . . . | 39        |
| <b>第五章 总结与展望</b>        | <b>39</b> |
| 5.1 主要工作总结 . . . . .    | 39        |
| 5.2 研究局限性 . . . . .     | 39        |
| 5.3 未来工作 . . . . .      | 40        |
| <b>参考文献</b>             | <b>41</b> |
| <b>附录</b>               | <b>42</b> |
| <b>致谢</b>               | <b>43</b> |





# 第一章 引言

## 1.1 研究背景与意义

近年来,协作机器人与自主移动机器人在工业制造、仓储物流、医疗服务等领域加速落地,成为推动产业智能化转型的核心载体。视觉—语言—动作 (Vision-Language-Action, VLA) 大模型将视觉感知与动作决策统一到端到端神经网络中,显著拓展了机器人在开放场景下的泛化能力。国家“十五五”规划 (2026—2030 年) 明确将具身智能机器人列为战略性新兴产业重点方向,标志着机器人从辅助性工具向智能制造和数字经济核心基础设施的历史性跃升。在此背景下,如何在资源受限的边缘平台上高效、可靠地运行机器人智能软件,已成为制约机器人大规模部署的关键问题。

从技术栈角度看,机器人系统通常由三个层次组成。**硬件层**涵盖上位机 (嵌入式计算板)、传感器 (摄像头、力传感器)、执行器 (电机、舵机) 与机械结构,决定机器人的物理感知与运动能力;**算法层**包括视觉感知、策略规划与运动控制,以 ACT<sup>[4]</sup>、Diffusion Policy<sup>[5]</sup> 等端到端模仿学习模型为代表;**基础软件层**则由操作系统 (Linux 等)、通信中间件 (ROS 等) 和推理框架 (PyTorch、ONNX Runtime、TFLite 等) 构成,向下管理硬件资源,向上支撑算法模型的运行。本文的研究聚焦于基础软件层:操作系统的内核调度策略、系统调用路径与设备驱动模型,以及推理框架的线程管理与算子调度,共同决定着机器人系统的硬件生态兼容性、推理性能、决策准确性、运行可靠性和系统安全性,是整个机器人软件栈性能与可信度的基础。

面向机器人基础软件的研究已有大量探索,但仍存在明显不足。广义上,机器人基础软件涵盖三个方向:操作系统内核、推理库、以及专用于机器人的通信工具。

在**操作系统内核**层面,现有路线可归纳为三类:以 Linux 为代表的**宏内核系统**应用广泛,但系统调用需经历用户态—内核态切换,单次操作引入百纳秒至数微秒的不确定延迟;以 HarmonyOS、Fuchsia 为代表的**微内核系统**结构精简、隔离性强,但驱动驻于用户态,硬件操作需经历两次态切换,整体延迟往往反而高于宏内核;以 FreeRTOS、Zephyr 为代表的**极简嵌入式实时系统**调度开销极低,但缺乏完整的深度学习推理支持,难以直接承载现代策略模型。

在**推理库**层面,PyTorch、ONNX Runtime、TFLite 各有侧重,均以“外挂式”方式接入操作系统:其内置线程池 (OpenMP 或 C10) 的调度与 OS 内核 CFS 调度器相互独立,在资源受限平台上容易引发调度冲突,造成额外的上下文切换与 futex 等待开销。

在机器人专用通信工具层面，ROS/ROS2 以 DDS 为底层通信层，micro-ROS 则将其下沉至嵌入式 MCU，二者均简化了多节点协作开发，但本质上继承了底层内核的调度局限，节点间消息传递延迟受制于 Linux CFS 调度器，无硬实时保证。

更根本的问题在于，现有性能分析工作多聚焦于单一层次——或模型压缩、或内核调度、或通信延迟——缺乏从 AI 推理框架层到 OS 内核层的跨层次联合剖析，难以回答“瓶颈究竟在哪一层、由什么原因导致”等关键问题。

针对上述研究缺口，**[TODO: 全文写完后再来改这段，需结合实验结论更新数据与贡献描述]**本文以 HuggingFace LeRobot 框架<sup>[3]</sup>下 ACT 策略在树莓派 5 (ARM Cortex-A76, 4 核, 4GB LPDDR4X, 无 GPU) 上的在线推理为研究对象，系统开展机器人基础软件的跨层次性能剖析。初步测试表明，单次 ACT 推理耗时约 1500—3000 ms，占“感知—推理—执行”循环总时间的 99% 以上；在 `chunk_size = 100` 的配置下，实际帧率仅约 18—19 fps，距 30 fps 的实时目标仍有明显差距。本文提出覆盖 AI 推理框架层、Python 运行时层、OS 内核层和硬件 I/O 层的四层性能分析模型，建立面向“感知—推理—执行”紧耦合场景的多工具联合剖析方法，通过 `torch.profiler`/`ftrace`/`perf` 多工具联合剖析，端到端量化各阶段时间开销，定位主要性能瓶颈，揭示 `strace` 带来的  $9\times$  时间放大问题并验证 `ftrace` 的必要性，为嵌入式机器人基础软件的性能优化与架构改进提供实验依据和工程参考。

## 1.2 国内外研究现状

机器人基础软件层面的研究可大致归为三个方向：操作系统、通信构件与开发 SDK；本节按此三个方向梳理国内外现有工作，并指出研究空白。

在操作系统层面，研究核心是**实时性保证**。标准 Linux 内核基于 CFS 公平调度，不提供硬实时保证；PREEMPT-RT 补丁通过把内核大段代码改为可抢占，将 ARM 嵌入式平台上的网络通信最坏延迟从 vanilla Linux 的  $13197\mu\text{s}$  压降至  $88-110\mu\text{s}^{\text{rtlinux}}$ ，是目前机器人控制系统最主流的 Linux 实时化方案；但仅有 PREEMPT-RT 还不够，并发流量下仍需 `cpuset` CPU 绑核隔离才能维持有界延迟<sup>rtlinux</sup>。ROS2 实时性研究综述<sup>ros2rtsurvey</sup> 进一步指出，内核 Executor 与定制调度器是该方向密集投入的研究分支。近期 AMS<sup>sams</sup> (Zheng et al., 2025) 将 action context、action exception、action replay 三个 OS 原语融入 VLA 推理管理，在真实机械臂任务上将成功率提升  $7\times-24\times$ ，执行步数减少 29%–74%，表明 OS 层机制对机器人推理的性能与可靠性具有直接影响。

在通信构件层面，ROS2<sup>ros2</sup> 以 DDS (Data Distribution Service) 为底层通信

层，是学术界与工业界的事实标准；但其节点调度仍依赖 Linux CFS，无硬实时保证。针对 ARM 嵌入式平台的实测表明，节点组合方式（intra-process vs. multi-process）对 CPU 占用与消息延迟有显著影响<sup>ros2composition</sup>；在树莓派 3/4 等平台上，Linux 网络栈被发现是制约硬实时以太网通信的主要瓶颈<sup>ros2ddsrpi</sup>。为支撑此类分析，ros2\_tracing<sup>ros2tracing</sup> 提供了基于 LTTng 的低开销追踪框架，可在不显著扰动主循环的条件下获取 ROS2 节点调度与消息传递的精细时序。

在 SDK 层面，深度学习推理框架是机器人基础软件最核心的组成之一。TF Lite<sup>tf lite</sup> 面向 MCU 级极端受限场景，支持 INT8 量化与 ARM NEON SIMD 加速；ONNX Runtime<sup>onnxruntime</sup> 通过 KleidiAI 算子库在 ARM64 上实现 28–51% 的性能提升；PyTorch 则提供 TorchScript 与 CPU 优化，是 LeRobot<sup>[3]</sup> 等机器人开源框架的默认推理后端。在机器人生态软件方面，HuggingFace LeRobot 等以 Python 为主的开源框架快速降低了 ACT<sup>[4]</sup>、Diffusion Policy<sup>[5]</sup> 等策略的训练与部署门槛，但围绕其性能特性的研究仍以模型精度与吞吐量为主要评估维度。更关键的是，上述推理框架与生态软件均以“外挂式”方式接入操作系统：其内置线程池（OpenMP 或 C10）的调度与 OS 内核 CFS 调度器相互独立，在资源受限平台上容易引发调度冲突，引入额外的 futex 阻塞与上下文切换开销，而这一类内核层耦合开销迄今缺乏系统性量化。

综上，已有工作多聚焦于上述三个层面中的**单一层面**：或针对内核做实时化改造、或评估通信中间件的延迟、或优化推理框架的算子性能；即便涉及多层（如 ros2\_tracing 联合用户态—内核态追踪），其覆盖范围也仅限于 ROS2 通信链路。**从 Python 运行时到 AI 推理框架再到 Linux 内核**这条贯穿机器人推理主循环的端到端调用链，迄今仍缺乏系统性的跨层次联合剖析。本文的研究方向正是**跨层次联合延迟分析**：建立覆盖 AI 推理框架层、Python 运行时层、OS 内核层与硬件 I/O 层的四层性能模型，通过 torch.profiler、ftrace、perf 等工具联合剖析，端到端定位 ARM 嵌入式平台上机器人推理主循环的真实瓶颈所在。

### 1.3 研究目标

本文以 LeRobot 为代表的机器人软硬件栈为研究对象，构建覆盖 AI 推理框架层、Python 运行时层、OS 内核层与硬件 I/O 层的跨层次性能分析框架，发现各阶段性能瓶颈，进行系统性量化测量，为后续性能优化提供实验依据。

## 第二章 实验环境构建

### 2.1 实验平台选型

本文选用 LeRobot<sup>[3]</sup>——HuggingFace 推出的开源机器人学习库,官方定位为”Making AI for Robotics more accessible with end-to-end learning”,提供从数据采集、模型训练到在线部署的完整工具链,支持 ACT、Diffusion Policy 等模仿学习策略。LeRobot 以单进程 Python 脚本串行执行感知—推理—执行三阶段,无分布式依赖;相比之下,基于 ROS2<sup>ros2</sup> 的方案以 DDS 为底层通信中间件,单次跨节点消息传递引入 1-5 ms 延迟,DDS 守护进程还额外占用 CPU 与内存<sup>ros2composition</sup>;第三章的剖析将表明通信开销远低于推理开销,引入 ROS2 只会带来不必要的测量干扰。

推理主机选用树莓派 5 (ARM Cortex-A76, 4 核, 4 GB LPDDR4X, 无 GPU), 搭配 SO-101 六自由度机械臂与两路 USB 摄像头。SO-101 是 LeRobot 配套的低成本开源机械臂,结构件可 3D 打印,具备完整的遥操作与部署支持。实验搭建了两个 SO-101 follower, 每个含 6 个舵机,由一块驱动板统一控制;在线推理实验只使用其中一个 follower, 两路摄像头(手眼与固定)均通过 OpenCV 采集。

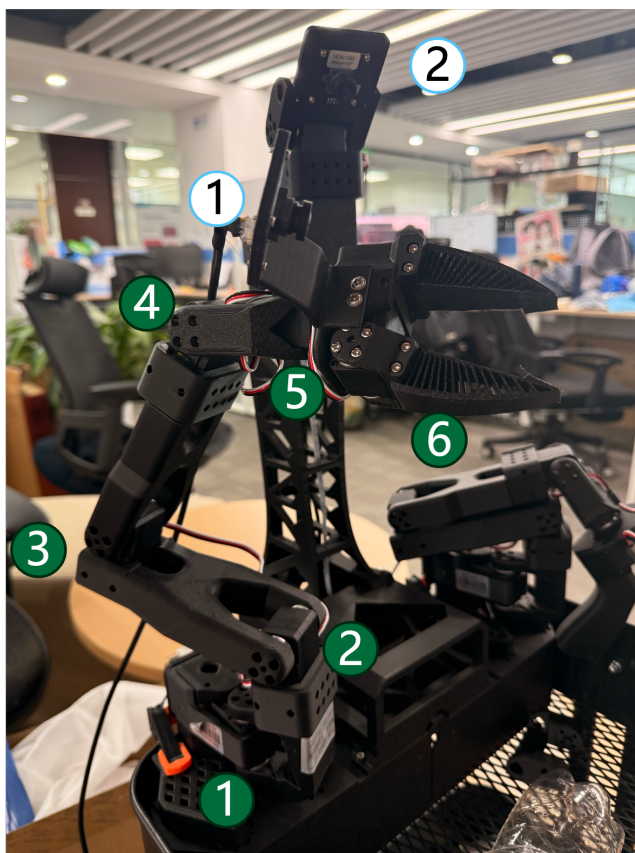


图 1: SO-101 六自由度机械臂实物 (标注关节编号 1-6)

| 编号 | 名称             | 说明                  |
|----|----------------|---------------------|
| 1  | shoulder_pan   | 底座水平旋转关节            |
| 2  | shoulder_lift  | 肩部俯仰关节              |
| 3  | elbow_flex     | 肘部弯曲关节              |
| 4  | wrist_flex     | 腕部弯曲关节              |
| 5  | wrist_roll     | 腕部滚转关节              |
| 6  | gripper        | 夹爪（末端执行器）           |
| C1 | handeye camera | 手眼相机，随臂运动，采集末端视角图像  |
| C2 | fixed camera   | 固定相机，俯视工作台，采集场景全局图像 |

策略模型选用 ACT<sup>[4]</sup> (Action Chunking with Transformers)，以 ResNet18 + Transformer 构成约 50M 参数的端到端模仿学习策略。以 RT-2<sup>[1]</sup>、OpenVLA<sup>[2]</sup> 为代表的 VLA 模型参数量达 7B+，在无 GPU 的 ARM 平台上几乎无法实时运行；Diffusion Policy<sup>[5]</sup> 参数量相近但扩散推理步数多导致延迟更高；ACT 是本文研究场景下可在树莓派 5 上实时运行的唯一可行选型。

| 项目    | 配置                                              |
|-------|-------------------------------------------------|
| 推理主机  | 树莓派 5, ARM Cortex-A76, 4 核, 4 GB LPDDR4X, 无 GPU |
| 机械臂   | SO-101 follower ×2; 推理时使用其中 1 个 follower        |
| 舵机与驱动 | 每个 follower 含 6 个舵机，由 1 块舵机驱动板控制                |
| 摄像头   | USB 摄像头 ×2，位置分别为 <b>handeye</b> 和 <b>fixed</b>  |
| 图像采集  | OpenCV Camera，分辨率 640×360，30 fps                |
| 控制参数  | <b>chunk_size=100</b> ，控制目标 fps=30，串口波特率 1 Mbps |
| 采集设置  | 每类任务采集 ≥ 3 次 episode 取均值，排除冷启动影响                |

| 软件/库              | 版本或说明           |
|-------------------|-----------------|
| 操作系统              | Raspberry Pi OS |
| Python            | 3.10            |
| LeRobot           | 0.3.4           |
| PyTorch           | 2.7.1           |
| torchvision       | 0.22.1          |
| OpenCV            | 4.12.0.88       |
| NumPy             | 2.2.6           |
| pyserial          | 3.5             |
| feetech-servo-sdk | 1.0.0           |

## 2.2 推理循环与软件栈

LeRobot 的在线推理是一个持续运行的帧循环：每一帧依次经历**观测采集**(`get_observation()`)、**策略推理**(`select_action()`)、**指令下发**(`send_action()`)和**等待**四个阶段，周而复始直到 episode 结束（图 2）。该循环在单个 Python 进程内串行执行，不依赖外部消息中间件，若以 30 fps 作为控制目标，则单帧可用时间约为 33.3 ms。

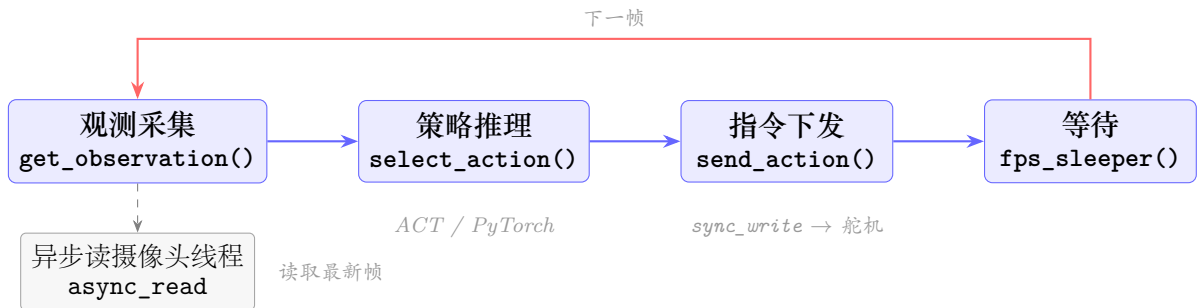


图 2: LeRobot 单帧推理循环流程：观测 → 推理 → 执行 → 等待，周而复始

每个阶段调用不同的软件栈路径（图 3）：**观测阶段**同时采集摄像头图像和关节角度——摄像头图像通过 OpenCV 后台线程异步预取，主线程调用 `async_read()` 取回已解码帧，底层经由 V4L2 驱动和内核 `pselect6` 等待帧就绪；关节角度由 `scservo_sdk` 通过 `pyserial` 以 `sync_read` 指令向舵机总线发起同步读取，底层经由串口驱动和内核 `read/write` 系统调用完成半双工通信。**推理阶段**将观测张量送入 ACT 策略网络，由 PyTorch（C10 线程池 + ARM NEON 算子）在 CPU 上完成前向计算，经由 `glibc` 和 Linux 内核的 `futex` 机制协调线程同步。**执行阶段**将预测动作通过 `scservo_sdk` 以 `sync_write` 写回舵机，路径与观测阶段的关节读取对称。**等待**

阶段由 `fps_sleeper` 调用 `time.sleep()` 补足帧间隔，底层触发 `nanosleep` 系统调用使进程主动让出 CPU。

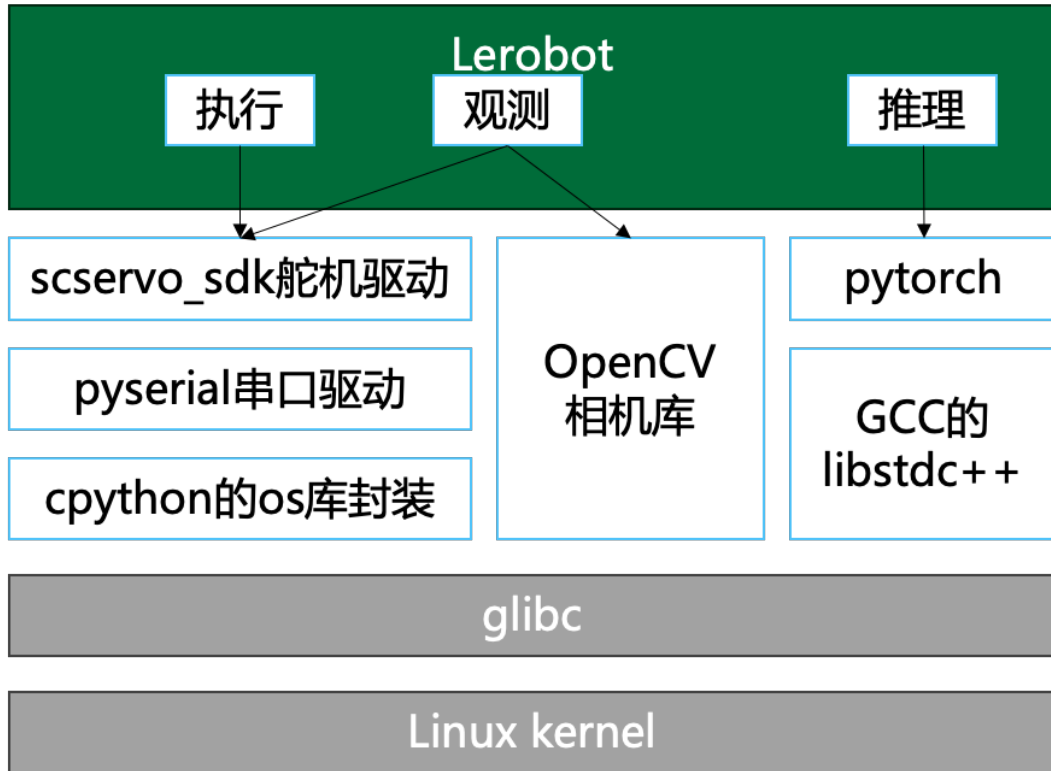


图 3: LeRobot 软件栈四层架构：应用逻辑层 → 库与驱动层 → glibc → Linux 内核

[来源：LeRobot 软件栈全链路分析.md]

### 2.3 三类典型工作负载特征

本文采用“遥操作采集—服务器训练—边缘推理”的标准流程构建实验数据集。数据采集阶段，操作者手持 leader 臂进行示教，SO-101 follower 臂实时同步复现动作，LeRobot 同步记录所有关节角度与两路摄像头图像，形成结构化 episode 文件。每类任务录制 50 个 episode，三类共 150 个 episode。训练阶段，将数据集传至配备 GPU 的服务器，调用 LeRobot `train` 命令以 ACT 策略进行监督学习，生成模型权重文件。推理阶段，将权重部署至树莓派 5，由 `record.py` 加载 policy 执行在线闭环推理，实验数据均在此阶段采集。

三类任务按操作复杂度从低到高设计，以考察不同难度下系统的推理行为是否存在差异：



| 难度 | 任务         | 描述                                        |
|----|------------|-------------------------------------------|
| 低  | 抓取 (Grasp) | 场景中放置 1 个瓶子，机械臂识别并完成抓取；目标唯一，动作路径固定        |
| 中  | 推倒 (Push)  | 场景中放置 2 个瓶子，需识别指定目标并将其推倒；引入目标选择判断         |
| 高  | 分类 (Sort)  | 场景中放置 3 个瓶子，需按指定顺序逐一抓取并分类摆放；需要长程规划与多步骤列操作 |

三类任务统一使用树莓派 5、ACT (`chunk_size=100`)、目标帧率 30 fps，每类任务取  $n = 50$  个 episode 的统计结果作为基准画像，只比较各阶段时间占比，而不把 episode 绝对时长差异混入分析。

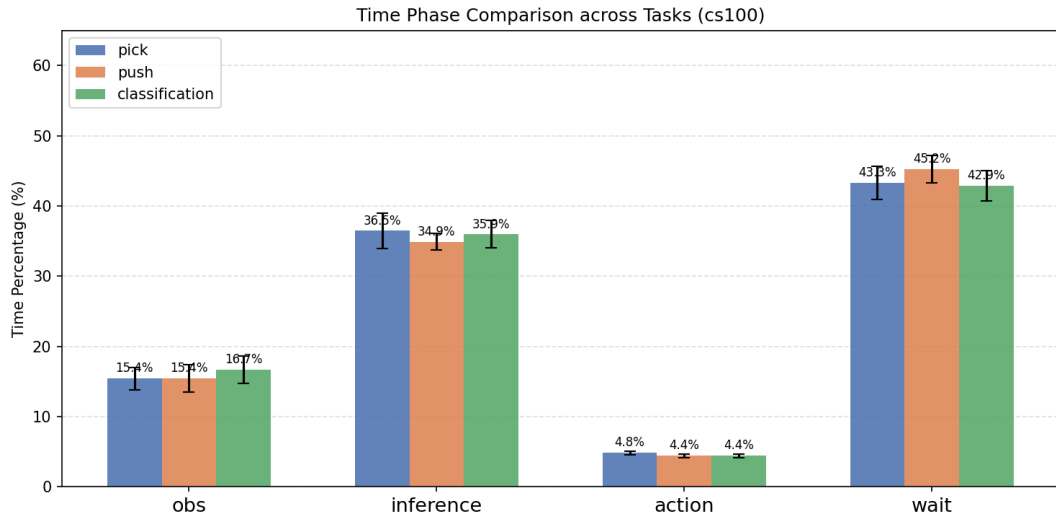


图 4: 三类任务在 `chunk_size=100` 下的阶段时间占比对比

图 4 的主要现象有三点。第一，三类任务的推理占比都集中在 35%–36% 左右，说明在模型结构、`chunk_size` 和输入配置相同的情况下，ACT 单次推理开销主要由模型本身决定，而不是由抓取、推倒、分类这些任务语义决定。第二，推倒任务的 `wait_pct` 最高，达到 45.2%，比抓取和分类高约 2 个百分点；这更像是动作行程和执行节奏造成的等待增加，而不是推理侧变慢。第三，`obs_pct` 和 `action_pct` 在三类任务之间变化很小，说明相机读取、舵机状态读取以及舵机写入在当前实验设置下更接近固定开销。

因此，后续性能分析不把三类任务视为三种完全不同的软件路径，而是把它们看

作同一 LeRobot 推理循环在不同动作负载下的表现。抓取和分类更接近基准工作负载；推倒任务由于等待占比最高，更适合用来观察执行节奏和 `chunk_size` 对整体帧率的影响。

[来源：实验/02\_ 三类任务工作负载对比/cs100\_task\_comparison.md]

## 第三章 性能分析

本章围绕 LeRobot 在线推理循环中的三个主要阶段展开：感知阶段负责采集两路相机图像和机械臂关节状态，推理阶段负责 ACT 策略前向计算，执行阶段负责将动作通过舵机总线下发到 SO-101 follower。三者虽然在 Python 主循环中串行出现，但其底层开销来源并不相同：摄像头路径主要经过 OpenCV/V4L2 和用户态图像解码，推理路径主要受 ARM CPU 上 PyTorch 算子计算限制，舵机路径则受半双工串口总线和 SCServo 协议约束。

### 3.1 工具链选型

**strace 与 ftrace 机制对比：**

两者作用层次不同，开销来源也不同：

- **strace**：通过 `ptrace` 系统调用在用户态拦截目标进程的每一次系统调用入口与出口。每次拦截都需要一次进程上下文切换（目标进程 → strace 进程），其开销与系统调用频率正相关，频繁调用时对主循环干扰极大，不能作为定量时序数据来源（具体扰动量级见后文实测）。
- **ftrace**：通过 Linux 内核 tracepoint 在内核态直接写入 ring buffer，插桩本身开销极低（纳秒级，无进程切换）。但将 ring buffer 内容导出至用户态文件（`cat /sys/kernel/debug/tracing/trace > log` 或后台进程持续读取 `trace_pipe`）是一次批量/持续文件 I/O，当 event 密集时，导出过程本身会消耗额外 CPU 和内存带宽，且实时导出期间 ring buffer 也可能覆盖旧数据（丢失事件）。

#### 测量工具开销实测

为定量比较各工具对主循环的扰动，本文设计了一个测量工具校准实验。在固定 `chunk_size=100`、目标 `fps=30`、episode 时长约 30 s 的同一条 `record_loop` 上，对比四组方案：A 组 baseline 不开任何 tracing；B 组用 `strace -f -ttT` 包住 `record` 命令；

C 组开启 ftrace 仅写内核 ring buffer, episode 结束后再导出 trace; D 组开启 ftrace 并同时启动后台进程实时读取 trace\_pipe 写入文件。所有指标由 record.py 内部通过 resource.getrusage() 与 sysfs 自动采集, 写入 analysis/timing\_stats.csv。表 1 汇总了四组的核心指标, 后文分别从 FPS、上下文切换次数和阶段占比漂移三个角度逐一分析。

表 1: 四组测量工具对 record\_loop 的扰动 (每组  $n = 1$ , 仅用于工具选型)

| 指标               | A baseline | B strace | C ftrace 延迟 | D ftrace 实时 |
|------------------|------------|----------|-------------|-------------|
| fps_mean         | 16.13      | 9.53     | 13.89       | 12.48       |
| 相对 A 的 FPS 下降    | —          | -40.9%   | -13.9%      | -22.6%      |
| system_time_s    | 1.40       | 2.76     | 1.37        | 1.03        |
| voluntary_cs     | 23,947     | 750,662  | 30,377      | 36,336      |
| involuntary_cs   | 78,436     | 541,334  | 84,481      | 52,907      |
| obs_pct 漂移 (pp)  | 0          | +34.8    | -5.5        | -3.0        |
| wait_pct 漂移 (pp) | 0          | -31.6    | -0.9        | -0.8        |
| trace 文件大小       | —          | 157 MB   | 168 MB      | 823 MB      |

**FPS 下降** 图 5 给出了四组的实测 fps\_mean 与相对 A 组 baseline 的下降幅度。B strace 组从 16.13 fps 跌至 9.53 fps, 下降 40.9%, 单这一项就足以排除 strace 作为正式时序数据来源的可能性。C ftrace 延迟导出组下降 13.9%, D ftrace 实时导出组下降 22.6%, 两者都仍然超过设计文档定义的  $\leq 5\%$  验收阈值; 但需要注意每组只有 1 个 episode, A 组 baseline 自身存在抖动 (如观测阶段单次冷启动时延变长会直接拉低绝对 FPS), 正式实验需补足每组 5 个 episode 后再核对方差。即便如此, C 组优于 D 组、且两组都明显优于 B 组的相对秩序非常稳定, 能够支撑后续的工具选型。

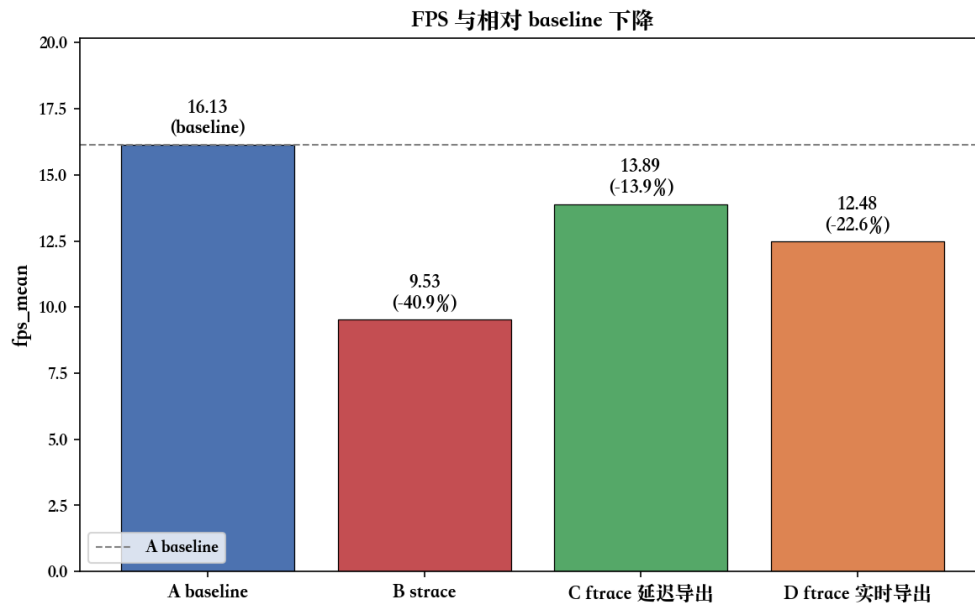


图 5: 四组测量工具下的 `fps_mean` 与相对 A 组 baseline 的下降百分比

**上下文切换次数** 图 6 用对数坐标对比了四组的自愿与非自愿上下文切换次数。B strace 组的 `voluntary_cs` 从 baseline 的约 2.4 万次飙升至约 75 万次（约 31 倍），`involuntary_cs` 也从 7.8 万次升至 54 万次（约 7 倍）。这种几何级放大是 `ptrace` 机制的直接体现：每一次系统调用都被强制中断到 strace 进程读取参数后再恢复，因此摄像头链路中密集出现的 `pselect6`、`read`、`ioctl` 都会在切换次数上留下印记。相比之下，C 组的切换次数仅比 baseline 高约 27%，D 组的额外切换主要来自后台 `cat trace_pipe` 进程，两者的量级都与 baseline 同档，说明 ftrace 内核 ring buffer 插桩本身对调度路径几乎无干扰。

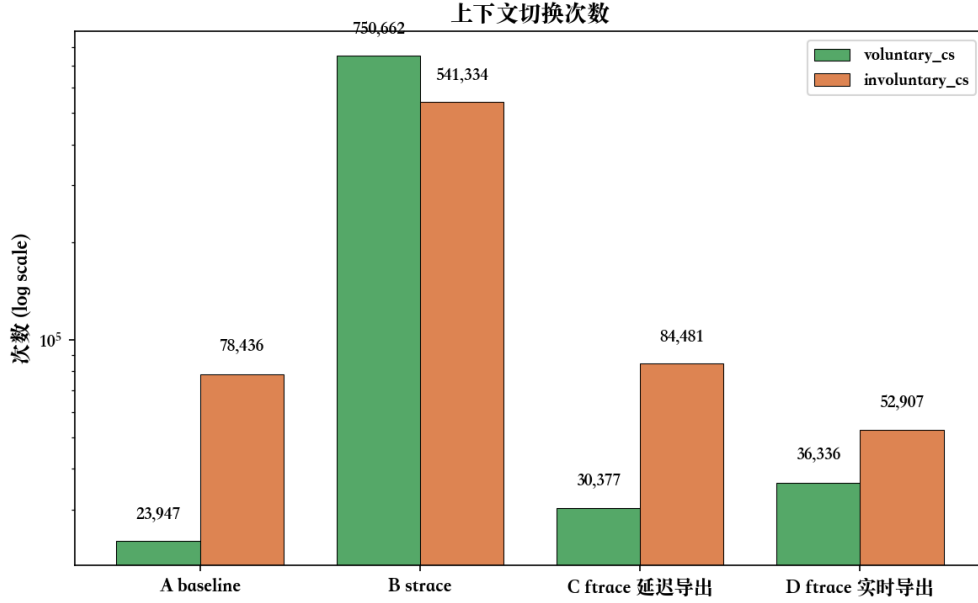


图 6: 四组的自愿与非自愿上下文切换次数（对数坐标）

**阶段占比漂移** 图 7 显示了四组各阶段占比相对 A 组 baseline 的偏移 (pp = percentage point)。B strace 组的 `obs_pct` 漂移高达 +34.8 pp, `wait_pct` 反向漂移 -31.6 pp, 意味着 strace 不仅整体拉慢主循环, 还改变了阶段时间结构: 摄像头链路中的系统调用被 `ptrace` 反复拦截, 导致观测阶段被严重拉长, `wait` 阶段反而被挤压; 这种结构性扭曲使得 strace 下采集的阶段占比已经无法代表真实负载。C 与 D 两组的所有阶段漂移都在  $\pm 8$  pp 内, 其中 `wait/action` 漂移均  $\leq 1$  pp, `obs_pct` 与 `inference_pct` 在  $\pm 7$  pp 之间小幅互相借调, 没有出现 B 组那种数量级的结构变化; 因此可以判断 ftrace 两种方案对阶段时间结构的影响可控。

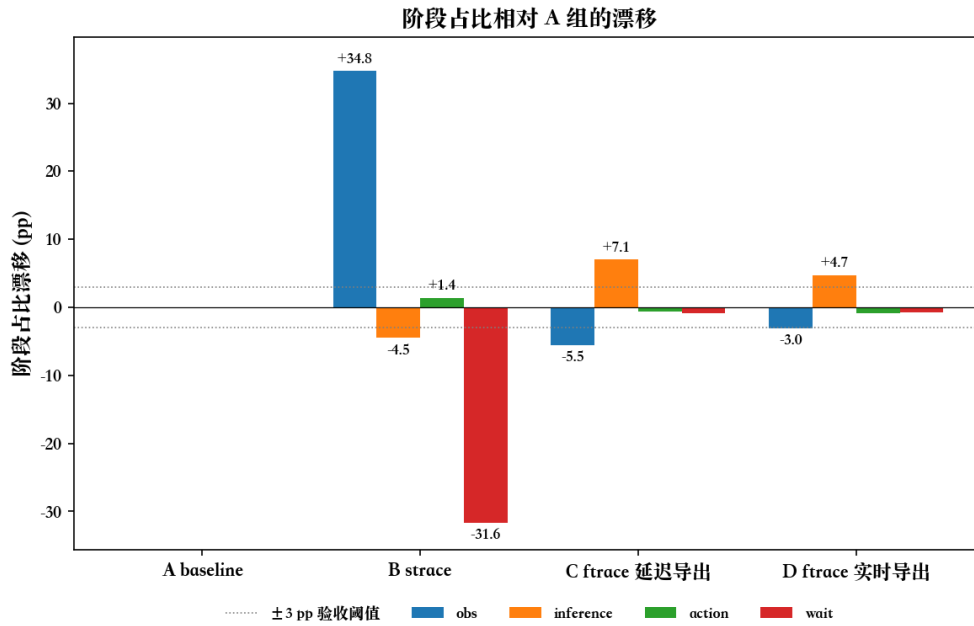


图 7: 四组的四阶段占比相对 A 组 baseline 的漂移 (pp), 灰色虚线为设计阶段提出的  $\pm 3$  pp 验收阈值

**本文工具链选型** 正式实验的内核态时序数据采用 **ftrace 延迟导出** 方案: 其上下文切换次数与阶段时间结构均与 baseline 同档, 是单 episode 扰动最小的方案。仅在 episode 时长延长导致 ring buffer (本文配置每 CPU 32 MB, 4 核共 128 MB) 接近溢出时, 才改用 ftrace 实时导出作为后备; 其代价是 trace 数据量约为延迟导出的 4.9 倍 (823 MB vs 168 MB), 但量级仍可承受。strace 不参与最终定量分析, 仅用于第三章早期对系统调用类型分布的定性探索。

**内核源码级插桩** 除上述标准 syscall tracepoint 外, 本文还在内核源码中通过 `trace_printk` 插入了自定义计时桩, 将更精细的时间拆分信息直接写入 ftrace ring buffer, 与标准 tracepoint 事件混合输出。标准 tracepoint 只记录系统调用入口参数和出口返回值, 无法拆分调用内部的阻塞睡眠时间与 CPU 处理时间, 也无法将文件描述符解析为设备名。

本文在两处关键路径上实现了自定义插桩:

1. **pselect6** (`fs/select.c`, `do_select()` 函数): 在函数入口通过 `ktime_get()` 记录起始时间, 在 `poll_schedule_timeout()` 前后打点计算阻塞睡眠时间, 函数退出时输出总耗时、睡眠时间、上下文处理时间、返回值、就绪 fd 编号及通过 `d_path()` 解析的设备名。例如, trace 中可直接观察到 “`fd=8, file=video0`” 表

示手眼摄像头帧就绪，“fd=9, file=video2”表示固定摄像头帧就绪，每次等待约 30 ms，睡眠时间约占总耗时的 99.7%。

## 2. `futex(kernel/futex/syscalls.c, do_futex())` 函数: 类似地在入口和 `futex_wait_multiple`

前后打点，输出操作码 (op)、返回值、总耗时、CPU 时间和睡眠时间。该拆分可以直接区分 `futex` 快速解锁 (`sleep≈0`) 与长时间等待 (`sleep` 接近 `total`)，从而判断 Python 线程间的锁竞争程度。实测数据显示 WAKE 操作 (`op=1`) 耗时在微秒级，而 WAIT 操作 (`op=0`) 的睡眠时间最高可达 1.8 s。

上述信息无法通过标准 `tracepoint` 或 `kprobe` 动态插桩获得：标准 `tracepoint` 不提供内部时间拆分；`kprobe` 回调中调用 `d_path()` 解析 `fd` 路径存在死锁风险，而 `do_select()` 原生上下文中调用是安全的。代价是需使用自编译内核，对本文固定的实验平台不构成额外负担。

[来源：实验/04\_ 实验环境与测量工具校准/结论.md，内核数据搜集分析.md]

## 3.2 感知阶段性能剖析

感知阶段对应主循环中的 `robot.get_observation()`。在本文的 SO-101 实验配置中，一次 `observation` 由两类数据组成：两路 OpenCV 摄像头图像，以及一个 follower 机械臂的 6 个舵机当前位置。其中两路摄像头分别为 `handeye` 和 `fixed`，分辨率均为  $640 \times 360$ ；舵机状态来自 6 个 STS3215 舵机的 `Present_Position` 寄存器。

从代码结构看，SO-101 follower 的观测函数先同步读取 6 个舵机位置，随后遍历两路相机，取回后台线程已经预取的图像。这意味着 `obs` 阶段并不是单一 I/O，而是“舵机同步读 + 两路相机异步取帧”的组合。舵机读取与执行阶段共用同一条 `/dev/ttyACM0` 半双工总线，协议和内核路径高度重合，因此本节只点明其在 `obs` 中的位置，详细分析放在 §3.4。

### 3.2.1 摄像头异步读取机制

LeRobot 的 OpenCV 相机并不是在主线程里每次直接阻塞读取硬件帧。每个相机对象内部维护一个后台读线程：只要没有收到停止信号，它就循环调用 OpenCV 读取摄像头，把最新图像写入共享缓存，并设置帧就绪事件。主线程中的 `async_read()` 不再直接面对摄像头设备，而是等待这个事件；事件到达后，它只需短暂获取锁，从共享缓存取走最近一帧，然后清除事件标志，等待下一次更新。这里不是两个线程互通知：后台线程负责生产帧并设置事件，主线程负责等待事件并清除事件。

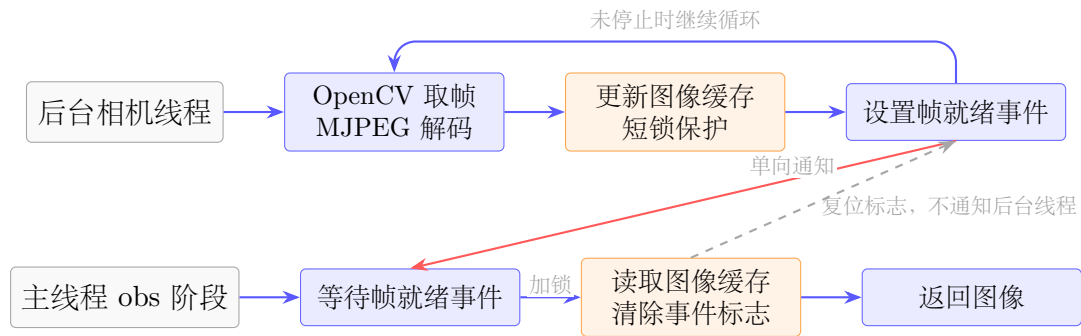


图 8: OpenCV 摄像头异步读取中的后台循环、单向事件通知与主线程取缓存关系

后台线程会调用 OpenCV 的读帧接口，从 V4L2 摄像头设备取出一帧，并在用户态完成 MJPEG 解码和颜色格式整理。整理后的 RGB 图像被写入 `latest_frame`，这个共享缓存由 `frame_lock` 保护。`new_frame_event` 只承担“帧已就绪”的单向通知作用；主线程取走缓存帧后会复位事件标志，避免下一轮直接读到旧帧，但这个复位动作不会反向唤醒或驱动后台相机线程。

因此，`async_read()` 的“异步”含义是：摄像头硬件读取和 MJPEG 解码提前在后台线程中进行，主线程在 `obs` 阶段只等待事件并读取最近一帧。如果后台线程已经准备好图像，主线程很快返回；如果主线程跑得比相机帧率更快，则会在等待新帧事件时阻塞。

### 3.2.2 是否进入内核

摄像头路径中有三类等待，需要区分其含义。

| 位置            | 是否可能进入内核 | 含义                                                                               |
|---------------|----------|----------------------------------------------------------------------------------|
| 主线程等新帧事件      | 可能进入内核   | 线程同步等待。对应 <code>new_frame_event.wait()</code> ，主线程等后台相机线程把新帧准备好                  |
| 主线程取缓存帧       | 通常不进入内核  | 短锁保护。对应 <code>frame_lock</code> ，只是在读写 <code>latest_frame</code> 时避免两个线程同时改同一块缓存 |
| OpenCV 后台线程取帧 | 会进入内核    | 设备可读等待。后台线程通过 <code>select()</code> 等待 V4L2 摄像头有帧可取，随后再执行出队操作取得该帧                |

也就是说，主线程的 `async_read()` 主要是在等 Python 线程同步事件；真正深入 V4L2、USB 摄像头驱动和 `videobuf2` 队列的是后台读线程。这也是为什么主线程



观测耗时可能只有毫秒级，而摄像头后台线程的 `pselect6` 等待会接近 30 fps 的帧间隔。

### 3.2.3 OpenCV 到内核的边界

论文正文不展开完整源码栈，只保留性能分析需要的边界关系：后台相机线程首先进入 OpenCV 的 `VideoCapture`，随后通过 `glibc` 发起 V4L2 相关系统调用，最终进入 Linux 的 V4L2、UVC 和 `videobuf2` 队列。

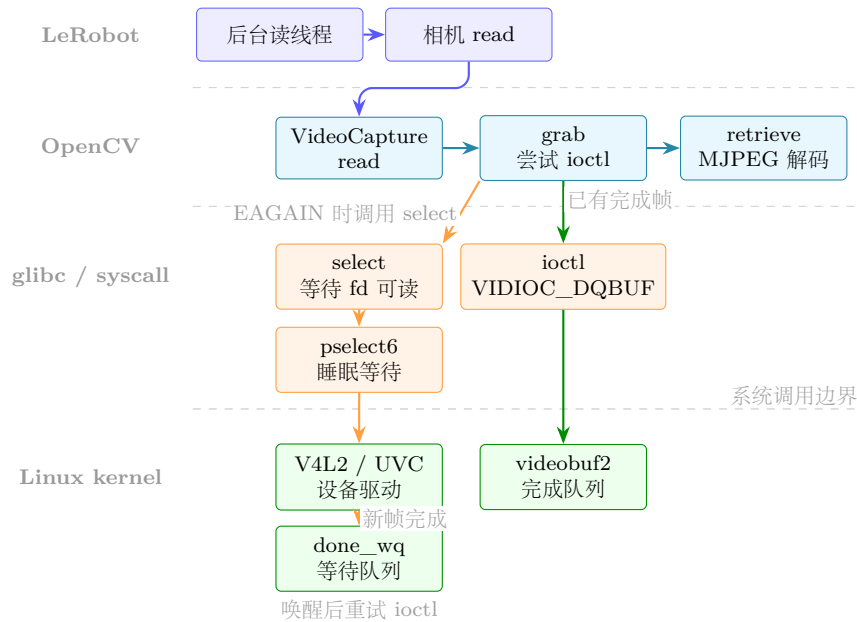


图 9: OpenCV 摄像头后台线程从用户态到 V4L2 内核队列的关键路径

图 9 中有两条需要区分的路径：等待设备可读通常发生在 `select()` / `pselect6` 一类调用上；真正取出已完成图像缓冲区时，OpenCV 使用 `VIDIOC_DQBUF` 将一个已经由驱动填好的 V4L2 buffer 从完成队列中出队。

需要注意的是，`VIDIOC_DQBUF` 并不是把整帧像素从内核重新拷贝一遍。在当前 OpenCV V4L2 路径中，摄像头 buffer 通过 `mmap` 暴露给用户态；出队操作主要返回 buffer 的编号、有效字节数和时间戳等元数据。MJPEG 到 BGR 图像的解码仍然发生在用户态 OpenCV/libjpeg 中，这部分才是 ARM CPU 上摄像头链路中更明显的计算开销。

完整的 OpenCV、glibc 和 Linux 内核源码级 CallStack 与参数说明，不放在论文正文中展开，单独记录在 `robotdoc/摄像头相关/OpenCV 到 glibc 到内核 CallStack 源码解析.md`。

### 3.2.4 时间剖析

在 30 fps 摄像头配置下，单路摄像头物理帧间隔约为 33.3 ms。实测中，后台线程在 `pselect6` 或 `V4L2` 等待路径中阻塞约 23–32 ms，符合摄像头帧率上限；MJPEG 解码仍在用户态 `OpenCV/libjpeg` 中完成，在 ARM CPU 上属于摄像头链路的主要计算开销。由于 LeRobot 采用后台预取，主线程 `obs` 阶段看到的是“取缓存帧”的时间，而不是完整的摄像头端到端采集时间。因此，主线程通常不会被摄像头硬件读取、`V4L2` 等待或 MJPEG 解码直接阻塞；这些耗时主要由后台相机线程承担。主线程只有在请求图像时后台线程尚未准备好新帧，才会短暂等待 `new_frame_event`，随后只是持锁读取最近缓存帧。

实测的逐层耗时分布与分层架构图见实验 06(摄像头感知层级耗时剖析, [robotdoc/实验/06\\_摄像头感知层级耗时剖析/](#))。该实验通过 `ftrace` 内核插桩和 Python `perf_counter` 双层时间戳，量化了从 `pselect6` 阻塞到 `tensor` 入模的层级耗时，并区分了“端到端时延”与“主线程感知耗时”在异步预取下的差异。

[来源: *OpenCV* 视频流内部实现.md, *OpenCV* 到 *glibc* 到内核 *CallStack* 源码解析.md, *pselect6* 最终结果.md, 实验数据/06\_摄像头感知层级耗时剖析/README.md]

## 3.3 推理阶段性能剖析

本节专门分析 SO-101 在线控制循环中的 ACT 策略推理阶段。这里的“推理阶段”并不只是一次 `ACT.forward()`，而是从 `predict_action()` 收到 `observation` 开始，经过动作缓存判断、必要时的观测预处理、ACT 模型前向、动作队列写入、再到动作反归一化返回的完整路径。

当前实验配置中，ACT 的 `chunk_size` 与 `n_action_steps` 均为 100，单次完整模型推理输出  $(B, \text{chunk\_size}, \text{action\_dim}) = (1, 100, 6)$  的动作序列。因此在线循环的大多数帧并不会重新执行模型前向，而是直接从 `_action_queue` 中取出上一次推理缓存的下一帧动作。这一区别是理解第 3.3 节所有时间数据的前提：`chunk_size` 主要是在时间上分摊单次 ACT 前向开销，而不是让一次 ACT 前向本身变快。

### 3.3.1 select\_action 与动作队列

LeRobot 在 `predict_action()` 外层先检查策略对象的动作队列。如果 `_action_queue` 非空，主循环直接 `popleft()` 取出缓存动作，再通过 `postprocessor` 反归一化为真实舵机目标；这一帧会跳过图像 `tensor` 转换、`preprocessor` 和 `ACT.forward()`。只有当队列为空时，系统才会走完整推理路径：把 `numpy observation` 转为 `PyTorch Tensor`，

执行归一化预处理,调用 `policy.select_action()`,继而触发 `predict_action_chunk()` 和 `ACT.forward()`。模型一次产生 100 帧归一化动作,写入队列后立即弹出第 0 帧返回;后续 99 帧由队列快速路径逐帧取用。

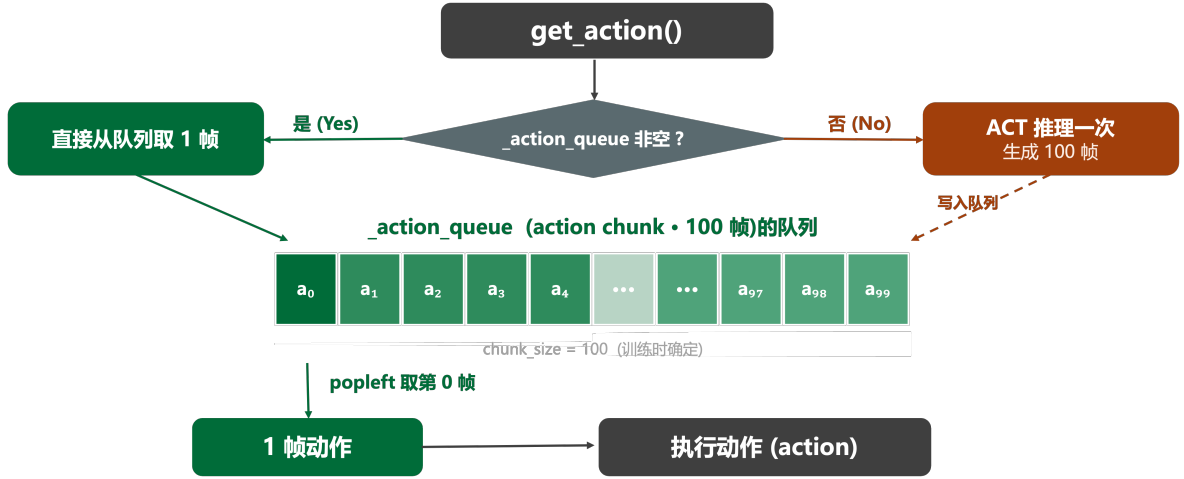


图 10: ACT 在线推理中动作队列快速路径与完整推理路径

图 10 说明了 `chunk_size=100` 下的两种时间尺度: 队列非空帧只承担动作取队列和反归一化, 属于轻量路径; 队列为空帧才承担完整 ACT 前向, 属于重推理路径。因此, 按 episode 统计得到的 inference 占比, 本质上是少数重推理帧在 100 帧执行周期内被均摊后的结果。

### 3.3.2 一次完整 ACT 推理的数据流

队列为空时, `predict_action()` 会先把原始 observation 整理成 ACT 可接受的 batch。两路相机图像从  $(H, W, C)$  的 `uint8` numpy 数组转换为  $(1, 3, 360, 640)$  的 `float32` Tensor; 关节状态从  $(6,)$  转为  $(1, 6)$ 。随后 `preprocessor` 使用随 checkpoint 保存的训练集 mean/std 对图像、state 进行归一化。进入 `ACT.forward()` 后, 推理态下不运行训练用 VAE encoder, 而是使用全零 latent。

图 11 中, 视觉分支是主要计算来源。每路  $640 \times 360$  图像经 ResNet18 的 `layer4` 得到  $512 \times 12 \times 20$  特征图, 两路相机共形成 480 个视觉 token; 再加上 1 个 state token 和 1 个全零 latent token, 构成长度 482、维度 512 的 Transformer Encoder 输入序列。Decoder 使用 100 个可学习 query, 最终通过线性层输出 100 帧、每帧 6 维的归一化动作。



图 11: 队列为空时一次完整 ACT 推理的数据流与模型结构

### 3.3.3 时间剖析：chunk\_size 参数扫描

实验 01 在树莓派 5 上对同一 ACT 抓取瓶子策略扫描 `chunk_size`，取值覆盖 {1, 3, 5, 10, 20, 50, 100}，每个取值固定记录 20 个 episode，其它参数（模型权重、分辨率、目标 fps）保持一致。原始数据与绘图脚本保存在 [来源：实验 01: `chunk_size` 参数扫描]。后文先看四个阶段的时间占比如何随 `chunk_size` 变化，再单独看 FPS 的增长曲线，最后用代表性数值表汇总。

**阶段时间占比随 `chunk_size` 的演化** 图 12 给出了 obs/inference/action/wait 四阶段平均占比（20 episode 均值）随 `chunk_size` 的变化曲线。`inference_pct` 从 `cs=1` 时的 97.6% 单调下降到 `cs=100` 时的 36.5%，对应 `wait_pct` 从 0% 单调上升到 43.3%；`obs_pct` 从近 0% 缓慢增长到 15.4%，`action_pct` 始终保持在 5% 以内。这条主曲线说明 `chunk_size` 越大，单帧分摊到的推理工作量就越小，控制循环把更多预算还给 `wait`（即在帧内等待 fps 节拍），推理不再压满整个 33.3 ms 时间预算。观测和执行阶段的占比上升只是相对结果——其绝对耗时基本恒定，只是因为 inference 缩短才在比例上变得显眼。

**FPS 随 `chunk_size` 的非线性增益** 图 13 给出了实测平均 FPS 随 `chunk_size` 的变化曲线。FPS 从 `cs=1` 的 0.8 上升到 `cs=100` 的 18.7，但增长曲线高度非线性：`cs=1→10` 时 FPS 提升约 6.6 倍（0.8→5.3），`cs=10→50` 时再提升约 2.9 倍，而 `cs=50→100` 时增益仅约 1.2 倍（15.4→18.7），呈现典型的边际递减。原因是单次完整 ACT 推理的

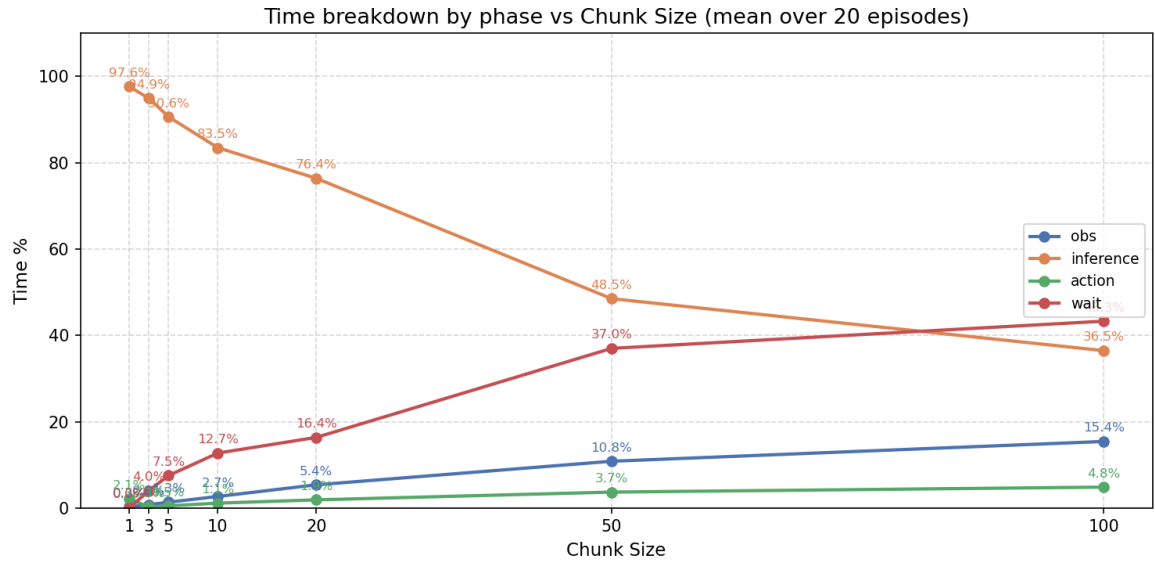


图 12: chunk\_size 对控制循环阶段时间占比的影响

绝对耗时接近恒定 ( $\approx 1-1.5$  s/次)，分块机制只是把这一次重推理在时间上摊薄到更多控制帧上，而不是降低 `ACT.forward()` 本身的计算量；当 `cs` 增大到推理被摊薄到接近 `fps` 目标时，等待阶段就成为主循环新的瓶颈，进一步增大 `cs` 不再带来等比增益。

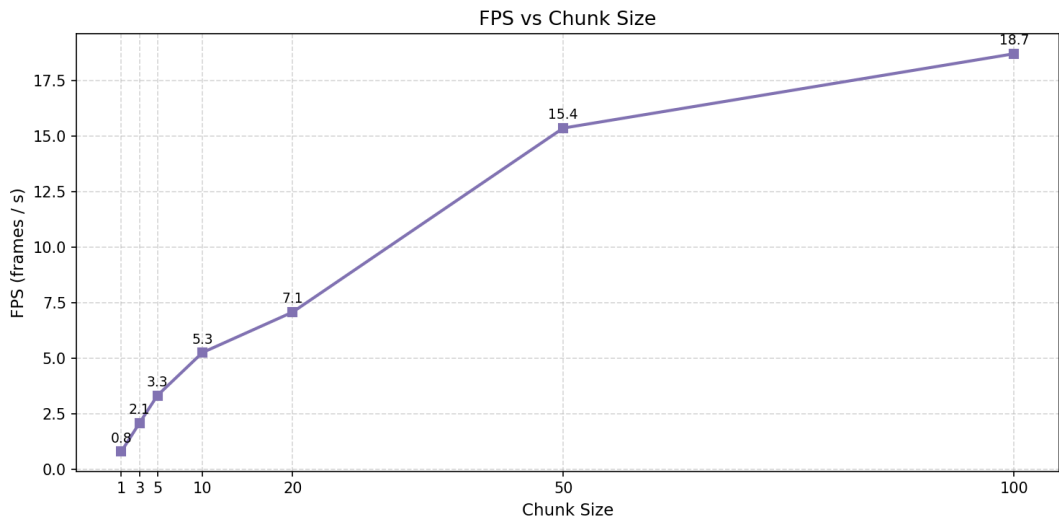


图 13: chunk\_size 对控制循环 FPS 的影响

`cs=100` 在树莓派 5 上能跑到  $\approx 18.7$  fps，距 30 fps 目标仍有约 38% 缺口；而此时 `wait_pct` 已升至 43.3%，说明继续增大 `cs` 已经不能进一步把 `fps` 推到 30——要逼近目标必须从减小单次推理绝对耗时入手，也就是后文（第四章）讨论的优化方向。

### 3.3.4 单次 ACT 推理耗时分解

上面的 episode 级统计能够说明 `chunk_size` 如何分摊推理开销，但还不能回答“一次完整 ACT 推理内部究竟慢在哪里”。实验在 `modeling_act.py` 的 `select_action()` 和 `ACT.forward()` 内插入计时点，通过环境变量 `LEROBOT_PROFILING=1` 启用，数据直接写入 CSV。在树莓派 5 上记录了 6 次队列为空的完整推理，结果汇总于表 2。

| 模块                              | 平均耗时 ms | 标准差 ms | 占完整推理比例 |
|---------------------------------|---------|--------|---------|
| <code>select_action</code> 完整路径 | 1924.8  | 414.9  | 100.0%  |
| ResNet18 + 投影                   | 1281.1  | 259.9  | 66.6%   |
| Transformer Encoder             | 569.5   | 506.3  | 29.6%   |
| Transformer Decoder             | 44.0    | 4.4    | 2.3%    |
| <code>action_head</code>        | 0.7     | 0.8    | <0.1%   |
| 队列非空快速路径（对照）                    | <1 ms   | —      | —       |

表 2: ACT 单次完整推理（队列为空时）的逐模块耗时分解

图 14 给出了各模块的耗时柱状图。

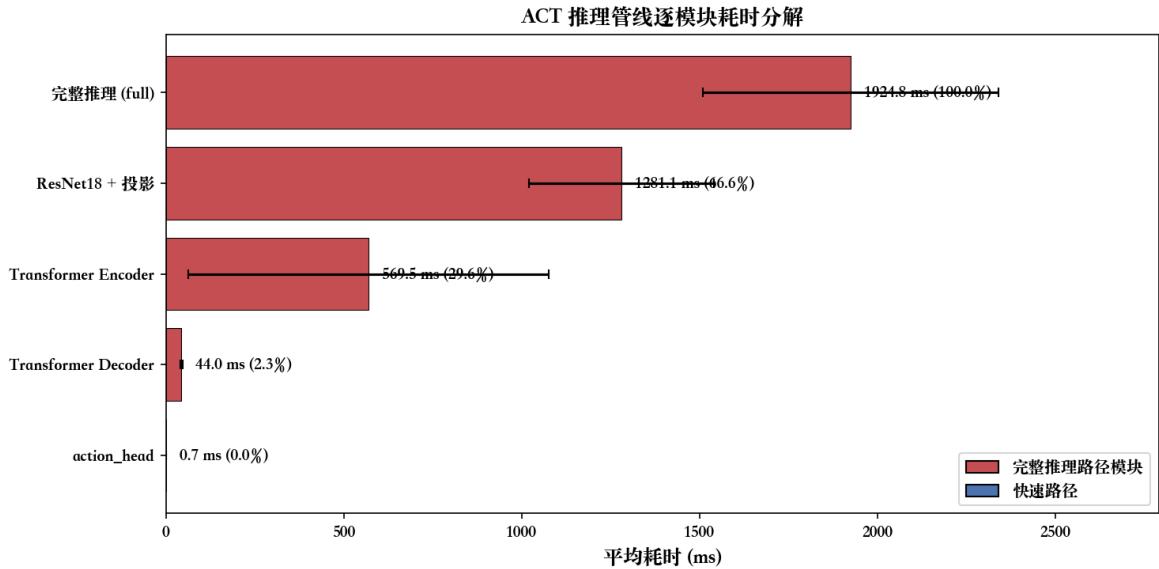


图 14: ACT 推理管线逐模块耗时分解

**关键发现：**ResNet18 视觉特征提取（两路 ResNet18 backbone + 位置编码 +  $1 \times 1$  投影）占据完整推理时间的 **66.6%**（平均 1281 ms），是第一瓶颈。Transformer Encoder 次之，占 29.6%（569 ms）。Decoder 和 `action_head` 占比极低（2.3% 和 <0.1%），不是优化重点。队列非空快速路径耗时不足 1 ms，与后处理相当，验证了动作缓存机制的有效性。

### 3.3.5 PyTorch 线程行为与推理优化研究现状

PyTorch 在 ARM CPU 上使用 OpenMP 并行框架，通过 ATen 线程池调度矩阵运算。系统调用追踪显示 futex 调用共计 64,569 次 / 40 s episode，其中 27 线程并行累计等待时间达 3,252 s，说明多线程同步开销在 ARM CPU 上不可忽略。

推理优化方向（如 INT8 量化、算子融合、线程亲和性、NEON SIMD 加速等）已有大量文献研究，此处不再展开。

## 3.4 执行阶段性能剖析

执行阶段的核心是机械臂六个 STS-3215 舵机的位置指令下发（`sync_write`）。为完整描述舵机链路，本节同时纳入感知阶段中物理通道完全重合的舵机位置读取（`sync_read`）。两者共用同一条 `/dev/ttyACM0` 半双工 TTY 总线、相同的 SCServo 协议帧结构与 USB 串口内核路径，仅在数据包内的指令字节（0x82 vs 0x83）和方向（写 vs 读 + 回包）上不同。

### 3.4.1 LeRobot 执行路径软件栈

图 15 展示了执行阶段从 `record.py` 的 `record_loop()` 到 `scservo_sdk` 的完整 Python 软件栈。

`get_observation()` 和 `send_action()` 分别对应读取舵机当前位置（`sync_read`）和写入目标位置（`sync_write`）的两条路径，共享 MotorsBus 层（归一化 / 符号编解码）和 Feetech 层（协议组包）。关键数据点：Present\_Position 寄存器地址 56，写 2 字节；Goal\_Position 寄存器地址 42，写 2 字节。读路径需等待 6 个舵机依次回包（半双工总线），写路径发完即返回，无回包。

### 3.4.2 `sync_write` 写链路：从电机总线到串口写入

图 16 展示了 `send_action()` 触发的完整写链路，横跨五个软件层：MotorsBus（归一化/编码）→ GroupSyncWrite（组包）→ GroupSyncWrite.syncWriteTxOnly()（协议帧）→ PortHandler（串口管理）→ `ser.write()`（内核写入）。

`_setup_sync_writer` 设置寄存器地址（`start_address=42`, Goal\_Position）与数据长度（2 字节），`addParam(id, data)` 将每个电机的目标位置序列化为 [lo, hi] 存入 `data_dict[id]`；随后 `makeParam()` 将 6 个电机数据拼成 16 字节负载，`syncWriteTxOnly()` 追加协议头与校验字节，构造出完整的 26 字节 SYNC\_WRITE 帧（FF FF FE 16 83 2A 02 [id lo hi] × 6 CS），广播 ID 0xFE 意味着无需等

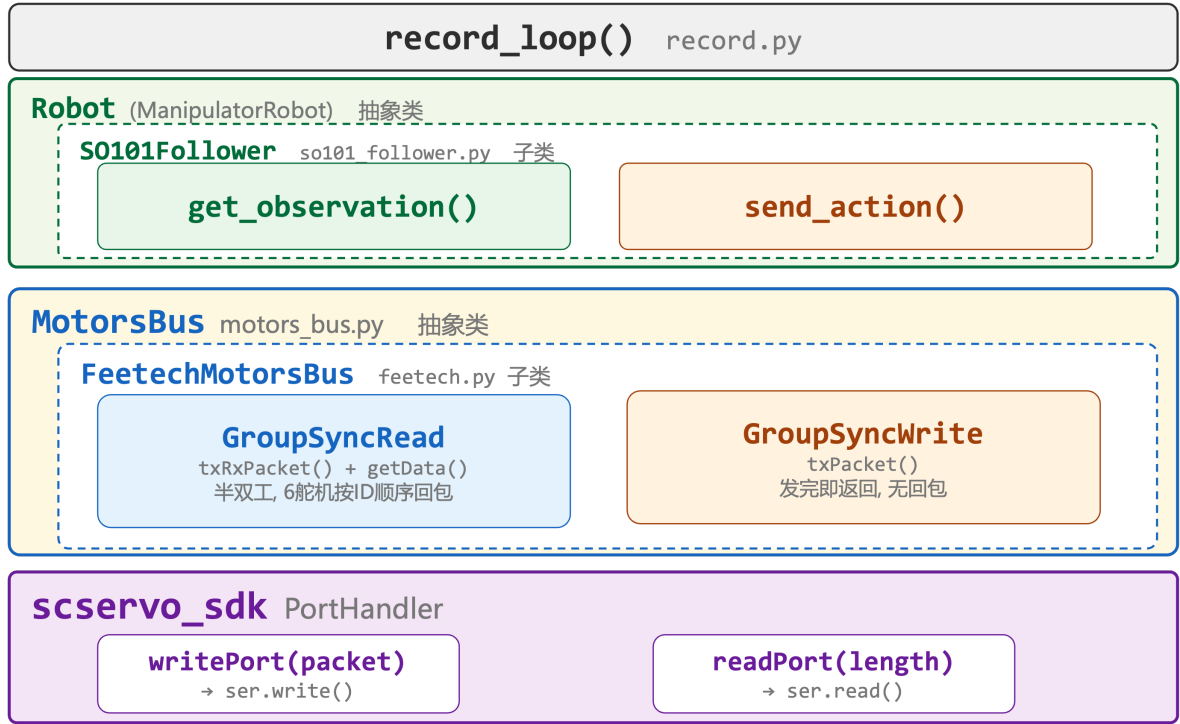


图 15: LeRobot 执行阶段 Python 软件栈调用链

待任何舵机回包；最终 `port.writePort(pkt)` 在 `is_using=False` 的条件下调用 `ser.write()` 将帧写入串口，写完即返回，无 RX 阶段，随后 `clearParam()` 重置组包缓冲区。

写链路的核心特性是“发完即返回”——广播帧不要求舵机回包，Python 侧耗时约 1–3 ms，瓶颈为 USB → UART 的物理传输，而非软件开销。

### 3.4.3 sync\_read 读链路：从请求帧到逐电机轮询回包

图 17 展示了 `get_observation()` 触发的读链路。与写链路共享前三层软件栈，但在 `PortHandler` 层之后分叉为完整的“发送请求帧 + 逐电机轮询”双阶段。

关键数据路径如下：

1. **请求帧发送**：`syncReadTx()` 构造 SYNC\_READ 帧（指令字节 0x82，寄存器地址 0x38/56，读长度 2），经 `ph.txPacket()` 调用 `writePort()` → `ser.write()` 写出。帧格式：FF FF FE 0A 82 38 02 01 02 03 04 05 06 CS。
2. **逐电机回包**：`rxPacket()` 对 `data_dict` 中每个 ID 依次调用 `readRx(port, id, 2)`，后者调用 `ph.rxPacket(port)` → `ser.read(n)`。`ser.read(n)` 内部以 while 轮询加超时退出，等待每个电机按 ID 顺序在半双工总线上依次发回状态帧（FF FF id 04 00 lo hi CS）。



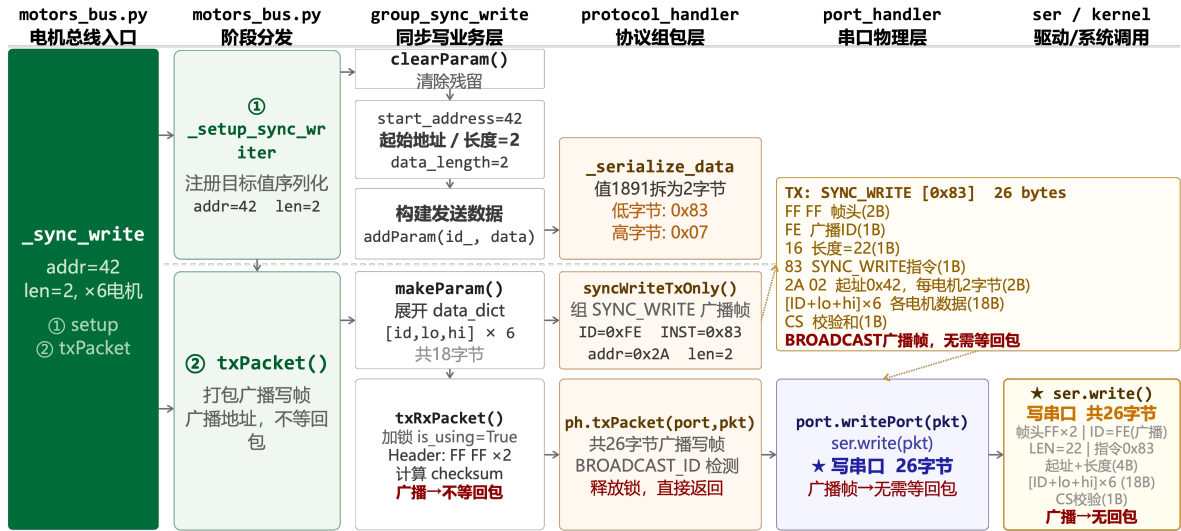


图 16: sync\_write 写链路: motors\_bus.py → GroupSyncWrite → ProtocolHandler → PortHandler → ser.write()

3. 数据解析: `getData(id, 56, 2)` 计算偏移量  $offset = 56 - 56 = 0$ , 取 `data[0]`, `data[1]` 合成 16-bit 位置值 (SCS\_MAKEWORD), 最终返回 {1: 2381, ..., 6: 1500} 形式的字典。

读链路耗时约 1.2 ms, 主要来源于 6 次串行回包等待。半双工总线在等待回包期间不能发送任何其他指令, 这是写链路无法与读链路并行化的物理根因。

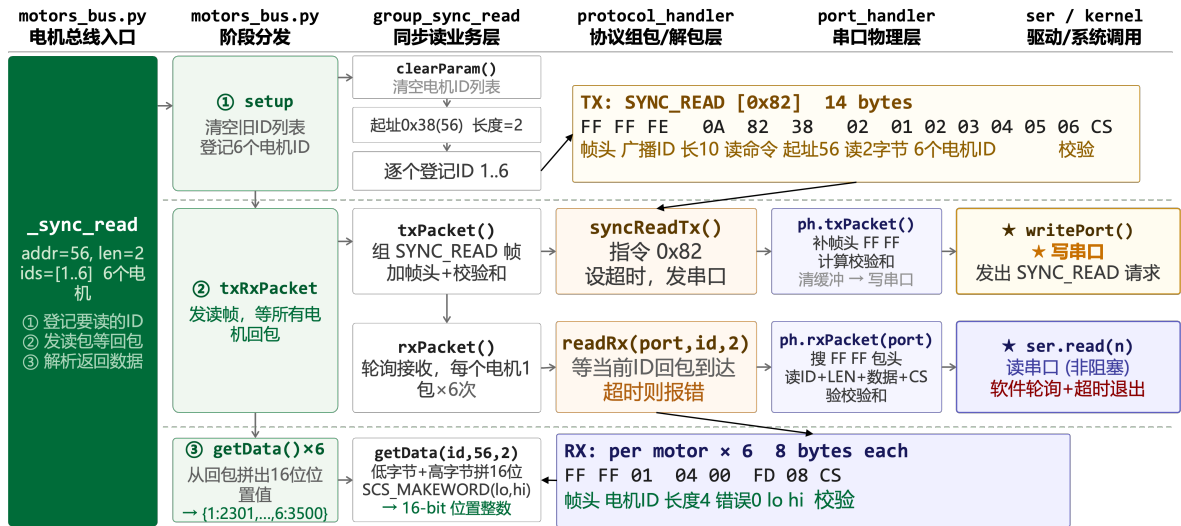


图 17: sync\_read 读链路: motors\_bus.py → GroupSyncRead → ProtocolHandler → PortHandler → ser.read() 轮询

### 3.4.4 关键性能结论

关键性能结论 (融合 sync\_read 与 sync\_write 两条路径的共性):

- **系统调用本身开销可忽略**：`write` 系统调用耗时仅  $38\ \mu\text{s}$ ，`read` 类似量级，二者组合（`sync_read`）约  $1.2\ \text{ms}$ ，单 `write`（`sync_write`）约  $1\text{--}3\ \text{ms}$ ；
- **瓶颈在物理层而非软件路径**：1 Mbps 波特率下，Sync Read 帧 14–15 字节、Sync Write 帧约 26 字节（ $6\ \text{ID} \times 4\ \text{字节位置} + \text{包头} + \text{校验}$ ），总线传输时间是耗时主体；
- **半双工约束是异步化的根本障碍**：同一时刻总线仅能容纳一个方向的传输，异步并发会导致总线冲突、指令帧损坏，且 `send_action()` 中 read-modify-write 的安全限幅链路依赖同步读返回的实时位置，异步化将破坏这一不变量（详见 §??）。

## 第四章 优化

### 4.1 优化方向的来源：从瓶颈结构反推

#### 4.1.1 第三章瓶颈结论回顾

第三章的逐阶段时序剖析得到一个清晰的结论：在树莓派 5 上，ACT 模型单次推理耗时  $2000\text{--}4000\ \text{ms}$ ，占总循环时间的 99% 以上；感知阶段（舵机 `sync_read` 加图像采集）合计仅约  $5\ \text{ms}$ ，执行阶段（舵机 `sync_write`）约  $1\text{--}3\ \text{ms}$ ，系统调用本身（`write/read/pselect6/futex`）最大量级在  $38\ \mu\text{s}$ 。也就是说，瓶颈不在 I/O、不在内核路径，而在用户态 ARM CPU 的矩阵乘法（GEMM）计算。本论文将研究范围限定在该低算力嵌入式平台，不讨论 GPU/NPU 加速等跨硬件优化路径。

#### 4.1.2 同步流水线下的 CPU 时间浪费

把第三章的耗时分布投影到时间轴上可以发现，当前 `record_loop()` 是一条严格串行的同步流水线：“采观测 → 推理一次 → 执行 chunk 内所有动作 → 再采观测”。在 `chunk_size=100`、目标 30 FPS 的设定下，执行 chunk 的 100 帧需要约  $3.3\ \text{s}$ ，而单次推理只占用 CPU 约  $2\ \text{s}$ 。也就是说，每一个“推理 + 执行”循环里有大约  $3.3\ \text{s}$  的窗口 **CPU 几乎完全闲置**（仅维持串口、相机后台线程和帧同步 sleep），其余 4 个 ARM Cortex-A76 核心都没有有效负载。

这段 CPU 闲置时间是**异步化的物理基础**：如果能在执行 chunk 的同时让 CPU 提前推理下一帧 chunk，理论上吞吐可以从单循环受限于“ $T_{\text{infer}} + N/F$ ”恢复到仅受

执行速度  $N/F$  约束，帧率有翻倍的潜在空间。

#### 4.1.3 三个候选异步点

把感知—推理—执行流水线展开来看，有三处理论上可以异步化的位置：

- **感知层——舵机异步读**：把 `sync_read` 拆成“发请求”与“收回包”两阶段，让两次相邻读取在时间上重叠；
- **执行层——舵机异步写**：把 `sync_write` 改为投递到后台队列，主线程立即返回；
- **推理层——双进程异步推理**：将 `policy.predict_action_chunk()` 移出主进程，主进程在执行旧 chunk 的同时让推理进程预测新 chunk。

值得说明的是，**相机异步读**已经是 LeRobot 的默认实现：`OpenCVCamera.async_read` 由后台线程持续 `cv2.VideoCapture.read()`，主线程取最新帧，本节不再讨论。

接下来 4.2–4.4 节按“可行性递进筛选”的顺序逐一审视上述三个异步点：4.2 论证舵机异步写在硬件协议层就被排除，4.3 论证舵机异步读虽然可行但 ROI 为负，4.4 把笔墨集中到唯一值得做的推理异步化，4.5 设计实验验证它在当前条件下的真实收益边界。

## 4.2 舵机执行通路异步化的可行性分析

### 4.2.1 半双工 TTL 总线的物理约束

机械臂六个 STS-3215 舵机走的是 Feetech SCServo 协议，物理层是单根 TTL 半双工串行线（`/dev/ttyACM0`）。“半双工”意味着同一时刻总线上只能存在一个方向的传输：要么主机向某个 ID 写指令，要么某个舵机向主机回报状态。六个舵机和主机控制板共享这一根线，所有事务必须严格串行。

这与摄像头的 USB 链路有本质区别。每台 USB 摄像头独占一条 USB 接口，后台 `async_read` 线程不停 `read()` 不会影响其他设备；而舵机只要后台线程发起任何写入，就会和主线程的 `sync_read` 在物理总线上撞车，造成字节流交错、回包校验失败、整帧数据废掉。

### 4.2.2 异步写为什么不可行

把 `sync_write` 改为后台队列异步执行，至少要面对四条独立的约束，任意一条都足以让该方案失败：

**约束 A——总线竞争：**异步写线程和主线程的 `sync_read` 不可避免地会同时尝试占用总线。即使引入互斥锁让两者轮流持锁，锁竞争和上下文切换会把异步带来的并发收益完全吃掉，退化成纯串行加额外开销，等于没异步。

**约束 B——安全限幅链断裂：**在 `so101_follower.py` 的 `send_action()` 中，当用户配置了 `max_relative_target` 时存在如下读改写链：

```
goal_pos = parse(action)
if max_relative_target is not None:
    present_pos = bus.sync_read("Present_Position")
    goal_pos = ensure_safe_goal_position(goal_pos, present_pos)
bus.sync_write("Goal_Position", goal_pos)
```

这条链严格要求“读到的当前位置”与“写出的目标位置”属于同一时刻，限幅基准才有意义。一旦写操作异步化，下一帧的 `sync_read` 可能读到上一帧目标尚未执行完的中间位置，`ensure_safe_goal_position()` 的限幅基准变得不确定，最坏情况下机械臂可能在限幅阈值附近发生连续误判。

**约束 C——错误暴露延迟：**当前 `_sync_write` 在通信失败时立即抛出 `ConnectionError`，调用方（`robot.send_action()`）能在同一帧内感知到“指令没送达”，可以停机、重试或上报。如果改为异步：主线程已经返回成功，`policy` 继续向前推理，几十毫秒后后台线程才发现包掉了，此时机械臂可能已经卡在某个半执行姿态，`policy` 还在按“执行成功”的假设输出下一条指令。在机器人安全场景里，这种错误反馈的滞后是不可接受的。

**约束 D——收益不在瓶颈上：**参考第三章的耗时分布，`sync_write` 写一帧约 1–3 ms，而 ACT 推理一次约 2000 ms。即便假定异步写能完美隐藏掉这 1–3 ms 的阻塞，对总循环时间的削减也不到 0.1%。在当前推理主导的负载下，任何 ROI 计算都会得到接近零的结果。

#### 4.2.3 小结

四条约束分别来自硬件协议 (A)、应用层不变量 (B)、安全工程实践 (C) 和工程 ROI (D)，任何一条都构成独立的否决理由。更关键的是，约束 A 和 B 是**结构性的**：只要硬件保持半双工 TTL 总线，只要 `send_action()` 还要做读改写式的安全限幅，异步写在本平台上都不存在落地空间。LeRobot 上游代码中既没有 `_async_write()` 占位方法、也没有任何 TODO 注释，也是出于同样的判断。因此本论文不再把舵机异步写视为候选优化方向。

### 4.3 舵机观测通路异步化的可行性分析

#### 4.3.1 上游 TODO：基于 txPacket/rxPacket 拆分的流水线方案

与异步写完全没有任何上游代码痕迹相对, `huggingface/lerobot` 主仓库的 `src/lerobot/motors.py` 在 `_setup_sync_reader` 与 `sync_write` 之间留有一段针对“读”的 TODO 注释占位：

```
# TODO(aliberts, pkooij): Implementing something like this could
# get even much faster read times if need be.
# Would have to handle the logic of checking if a packet has been
# sent previously though but doable.
# This could be at the cost of increase latency between the moment
# the data is produced by the motors and the moment it is used by
# a policy.
# def _async_read(self, motor_ids, address, length):
#     if self.sync_reader.start_address != address or ...:
#         self._setup_sync_reader(motor_ids, address, length)
#     else:
#         self.sync_reader.rxPacket()
#         self.sync_reader.txPacket()
#     for id_ in motor_ids:
#         value = self.sync_reader.getData(id_, address, length)
```

注释明确指向一种“流水线 (pipelining)”方案：把当前 `txRxPacket()` 中的“发请求”与“收回包”两个原本绑在一起的步骤拆开，让每次 `_async_read()` 调用先接收上一次请求的回包，再发起这一次的新请求，下次调用再来收这一次的回包。本质上是把单次串口往返的等待时间隐藏到两次相邻调用之间。

#### 4.3.2 与相机后台线程式异步的本质区别

需要特别澄清：上游规划的“异步读”与相机的 `async_read` 完全不是一个机制。

- 相机 `async_read`：由独立后台线程持续 `cv2.VideoCapture.read()`，主线程仅取最新帧。属于多线程并发模式，受益于每台相机独占 USB 端口，没有总线竞争问题。

- 舵机 `_async_read` (TODO): 完全在主线程内执行, 没有任何后台线程。仅仅是把单次原子的 `txRxPacket()` 拆成两次连续调用之间的“收 + 发”重叠。属于串行流水线模式, 因此不会与 `sync_write` 在总线上撞车。

也正因为是流水线而非并发, 4.2 节中关于半双工总线竞争 (约束 A) 和错误暴露延迟 (约束 C) 的论证对舵机异步读并不适用: 异步读不引入并发, 不需要锁, 错误依然在主线程当帧抛出。

#### 4.3.3 落地代价: 驱动改造与观测滞后

尽管异步读在原理上可行, 要真正落地仍有两项代价:

**代价 1——需要修改 `scservo_sdk` 驱动层。**当前 Feetech SDK 把 `txPacket()` 与 `rxPacket()` 的调用封装在 `txRxPacket()` 里作为单个原子操作, 对外只暴露这一接口。要落地上游 TODO 的 pipelining 方案, 必须深入 `scservo_sdk` 内部, 把内部的 tx/rx 状态机暴露给 `motors_bus.py`, 还需新增“上次发送的请求参数”缓存以及“参数不匹配则退化为同步读”的兼容路径。这一改造涉及一个属于第三方厂商的 C/Python 混合驱动, 维护与上游升级跟踪都会有额外负担。

**代价 2——观测延迟 1 帧。**TODO 注释明确写了: “This could be at the cost of increase latency between the moment the data is produced by the motors and the moment it is used by a policy.” 异步读在第  $N$  次调用拿到的实际上是第  $N - 1$  次请求时的电机状态。对 ACT 这类闭环 policy 而言, “此刻的观测”比物理时刻晚了一整帧 (约 33 ms @30 FPS), 理论上会让控制误差略增。

#### 4.3.4 收益估算

按第三章的实测数据, 单次 `sync_read` 的串口往返耗时约 1.2 ms。异步读最多能把这 1.2 ms 从主线程的关键路径上拿掉。然而:

- 在当前 ACT 主导的循环中 (推理约 2000 ms), 节省 1.2 ms 对总循环时间的影响小于 0.1%;
- 即便后续推理被加速到 100 ms 量级 (INT8 + ONNX), 异步读省下的 1.2 ms 也仅占约 1%, 仍属边际收益;
- 真正可能让异步读发挥作用的场景, 是在没有视觉、没有大模型推理的纯实时控制系统里 (控制频率 500 Hz 以上), 而这类场景与本论文的 LeRobot ACT 用例并不重合。

#### 4.3.5 小结

舵机异步读在原理上可行 (pipelining 不破坏半双工约束), 但综合来看 ROI 不足以支持改造工作量: 当前推理瓶颈下省下 1.2 ms 几乎不可见, 且需要改第三方 SDK 并接受 1 帧观测滞后。本论文将其归入“结构上可行、当前不值得做”的优化项, 作为推理加速完成后可以重新评估的潜在方向, 但不在本论文的实验范围内。

### 4.4 推理异步化: 本章重点

#### 4.4.1 设计动机: 让推理与执行真正并行

回到 4.1.2 节指出现象: 在同步流水线下, 每个“推理 + 执行 chunk”循环里有 3.3 s 的 CPU 闲置窗口。如果能把下一帧的推理重叠到当前 chunk 的执行期里, 吞吐有望从“ $T_{\text{infer}} + N/F$ ”—限制 (约 19 FPS) 恢复到仅受执行频率约束的 30 FPS。这就是推理异步化的核心设计动机。

与 4.2 / 4.3 节讨论的舵机异步化不同的是: 推理过程纯粹是 Python+PyTorch 计算, 不与任何半双工硬件资源耦合, 理论上具备真正并行的物理基础; LeRobot 上游也已为此提供了完整的双进程实现, 无需自行从零开发。因此本节的工作不再是“评估能不能做”, 而是“评估在树莓派 5 + 当前 ACT 模型的具体参数下, 做了能不能带来实际收益”。

#### 4.4.2 LeRobot 官方双进程架构

LeRobot 把推理异步化拆成两个独立进程, 通过 gRPC 通信:

- `robot_client.py`: 主进程, 连接硬件, 跑控制循环, 采观测发给 server, 从 server 收回 `action_chunk` 后按帧发给舵机执行;
- `policy_server.py`: 推理进程, 加载 ACT 模型权重, 收到观测后运行 `policy.predict_action`, 把动作序列序列化 (pickle) 后回传。

两进程之间的关键数据载体是 `TimedAction` 与 `TimedObservation` (见 `helpers.py`), 都包含 `timestamp` 和 `timestep` 字段, 分别用于跨进程网络延迟测量和帧号对齐。

在 `robot_client` 内部, 主线程跑控制循环、后台线程 (`receive_actions`) 阻塞等待 server 回包; 两个线程通过带锁的 `action_queue` (`queue.Queue`) 共享数据。关键状态变量包括:

- `latest_action`: 已发给舵机的最新帧号, 初始为 -1;

- `action_chunk_size`: server 每次返回的 chunk 长度，首次收到 chunk 后更新；
- `must_go`: `threading.Event`，队列即将耗尽时强制 server 立即推理。

#### 4.4.3 三个核心机制

**机制一——`chunk_size_threshold`(提前推理):** client 主循环每帧检查 `action_queue.qsize()` / `action_chunk_size` 是否已经  $\leq$  `chunk_size_threshold` (默认 0.5)。即队列剩 50% 时就开始采下一帧观测、向 server 发推理请求，而不是等队列彻底空了才开始。设计意图是让“新 chunk 到货的时刻”正好接上“旧 chunk 耗尽的时刻”，避免主循环 stall。

**机制二——Temporal Ensemble (多 chunk 加权融合):** 新 chunk 到达时不是直接覆盖队列，而是按 `timestep` 对齐。对于“新旧 chunk 都覆盖到的同一未来帧”，使用用户配置的 `aggregate_fn` (默认取新值，可改为加权平均) 做融合；对于“新 chunk 覆盖到、但已经被主线程执行过”的帧 (`timestep \leq latest_action`) 则直接丢弃；对于“纯未来帧”则入队。这是 ACT 原论文提出的 temporal ensemble 在异步场景下的自然实现，用于减少 chunk 切换时的动作抖动 (jitter)。

**机制三——`must_go` (队列空兜底):** 如果推理慢到队列真的空了，client 在采观测时把 `must_go=True` 随观测一起发给 server；server 端跳过所有去重检查，强制把这次观测放入推理队列；client 收到新 chunk 后再 `must_go.set()` 重置。这条路径属于异常兜底，正常稳态下不应频繁触发——`must_go` 触发频次因此是判断流水线健康度的重要指标 (4.5 节实验会重点采集)。

#### 4.4.4 可行性不等式 $N \geq 2T_{\text{infer}}F$ 的推导

异步推理的稳态行为受三个参数控制：`chunk_size`  $N$ 、单次推理耗时  $T_{\text{infer}}$ 、目标执行频率  $F$ 。本节从单次推理产出的“有效帧数”出发推导稳态条件。

设主线程在第 0 帧完成对当前 chunk 的耗尽前 50% (即触发阈值)，开始采观测发给 server。server 推理耗时  $T_{\text{infer}}$ ，这段时间内主线程消耗  $T_{\text{infer}} \cdot F$  帧。当新 chunk 到货时，chunk 内的前  $T_{\text{infer}} \cdot F$  帧已经过期 (其 `timestep \leq latest_action`)，在 client 端被直接丢弃；真正能进入队列、被未来执行的有效帧数为：

$$N_{\text{eff}} = N - T_{\text{infer}} \cdot F. \quad (1)$$

下一次推理周期需要主线程消耗另外  $T_{\text{infer}} \cdot F$  帧才能再次到货，因此稳态不 stall



的条件是“一次推理产出的有效帧数  $\geq$  下一次推理期间消耗的帧数”：

$$N - T_{\text{infer}} \cdot F \geq T_{\text{infer}} \cdot F \iff \boxed{N \geq 2 T_{\text{infer}} F}. \quad (2)$$

为直观展示该不等式的物理含义，图 18 给出  $N = 100$ 、 $F = 30$  时三种代表性  $T_{\text{infer}}$  取值下的稳态时间线（一格 = 10 帧； 表示正常执行， 表示 stall）。

情况 A:  $T_{\text{infer}} = 30$  帧 (1 s) 不等式满足，稳态 30 FPS  
 执行：  
 推理： P1 (30 帧) P2 (30 帧)

情况 B:  $T_{\text{infer}} = 50$  帧 (1.67 s) 临界点，无 ensemble 余量  
 执行：  
 推理： P1 (50 帧) P2 (50 帧)

情况 C:  $T_{\text{infer}} = 70$  帧 (2.33 s) 不等式不满足，周期性 stall  
 执行：  
 chunk A 100 帧 stall 20 帧 chunk B 50 帧 ...

图 18:  $N = 100$ 、 $F = 30$  时三种  $T_{\text{infer}}$  下的稳态时间线

值得指出的是情况 C 下的一个微妙之处:stall 期间机器人停止执行,latest\_action 被冻结，因此当新 chunk 到货时，“过期帧”的判定基准并不是“物理时刻已经走过的帧数”  $T_{\text{infer}} \cdot F$ ，而是 stall 之前最后一次执行到的帧号。也就是说，stall 的那段时间“反过来保护”了新 chunk 的有效帧不被丢弃。具体到情况 C，新 chunk 时间戳 [50, 149] 到货时 latest\_action = 99 (chunk A 最末帧)，有效帧数 = |[100, 149]| = 50 而非 30。这一点必须在公式推导中保留，否则时间线与公式会出现不自洽。

#### 4.4.5 不等式在本平台的代入

代入第三章的实测参数  $T_{\text{infer}} = 2$  s、 $F = 30$  FPS：

$$2T_{\text{infer}}F = 2 \times 2 \times 30 = 120 > 100 = N. \quad (3)$$

不等式不满足。理论稳态 FPS 退化为：

$$F_{\text{actual}} \approx \frac{N - T_{\text{infer}}F}{T_{\text{infer}}} = \frac{100 - 60}{2} = 20 \text{ FPS}, \quad (4)$$

与同步模式的实测约 19 FPS 几乎持平。也就是说在当前参数下，异步推理理论上几乎不会带来吞吐提升，反而会引入双进程、gRPC 通信、锁竞争和 CPU 争抢等额外复杂度。这一理论预测正是 4.5 节实验要验证的对象。

#### 4.4.6 树莓派 5 单机部署的 CPU 争抢

异步推理在理论分析之外还有一项实操层面的注意点：树莓派 5 只有 4 个 ARM Cortex-A76 核心。ACT 推理通过 PyTorch 的 OpenMP 后端默认会吃满所有核心，而控制循环、串口 IO、相机解码也需要约 1 核。如果 `policy_server` 和 `robot_client` 同机部署，两者会在核心层面互相挤压，推理线程被频繁让出 CPU 导致  $T_{\text{infer}}$  抬高。

为隔离两进程对 CPU 的争抢，4.5 节实验将采用以下控制措施：

- `taskset -c 0-2` 把 `policy_server` 钉在核心 0-2；
- `taskset -c 3` 把 `robot_client` 钉在核心 3；
- 在 `policy_server` 内显式 `torch.set_num_threads(3)`；
- 在 `robot_client` 内 `cv2.setNumThreads(1)` 避免 OpenCV 内部多线程与控制循环抢 CPU。

需要承认，3 核推理相对 4 核独占大约慢 25%， $T_{\text{infer}}$  会相应抬高，进一步压缩不等式的理论余量。这是单机部署的固有代价，也是 4.5 节实验中需要单独测量的量。

### 4.5 异步推理收益的实验验证

#### 4.5.1 实验目的与论证逻辑

4.4.5 节的不等式分析给出一个具体可证伪的预测：在当前  $T_{\text{infer}} \approx 2\text{ s}$ 、 $N = 100$ 、 $F = 30$  的条件下，异步推理理论稳态 FPS  $\approx 20$ ，与同步模式（约 19 FPS）几乎无差异。本节设计实验对这一预测进行直接验证。

需要说明的是：实验的目的**不是证明异步推理普遍有效**，而是用实测数据为“异步推理在当前参数下不带来收益”这一否定性结论提供工程证据。这种可行性边界的定量刻画本身具有工程价值——它能避免在错误的优化路径上投入工时，也为“推理本体加速到什么程度后再启用异步”这一后续工作提供清晰的判据。

#### 4.5.2 实验设计

实验采用单变量对照，自变量为“调度模式”，其余参数均严格控制以排除干扰：

| 维度                   | 取值               | 备注                                    |
|----------------------|------------------|---------------------------------------|
| 调度模式                 | {同步, 异步}         | <b>唯一自变量</b>                          |
| chunk_size $N$       | 100              | 与第三章同步 sweep 对齐                       |
| $T_{\text{infer}}$   | $\approx 2$ s    | 原始 ACT, 不做量化或加速                       |
| 目标 FPS $F$           | 30               | <code>environment_dt = 33.3 ms</code> |
| chunk_size_threshold | 0.5              | 异步默认值                                 |
| 任务                   | 同一段录制好的回放任务      | 避免在线遥操污染                              |
| 每模式重复                | $\geq 3$ episode | 每段 $\geq 30$ s, 弃前 5 s warm-up        |

#### 4.5.3 控制变量

为保证两种模式之间的差异只来自调度方式，实验严格固定以下条件：

- CPU 频率锁定为 `performance governor`，防止 DVFS 节流污染时序；
- `torch.set_num_threads(3)` 与 `cv2.setNumThreads(1)` 在两种模式下保持一致；
- 异步模式下 client 和 server 同机部署，通过 `taskset` 把 server 绑核到 0-2、client 绑核到 3，与 4.4.6 节描述一致；
- 每个 episode 前后用 `vcgencmd measure_temp` 记录 CPU 温度，记录每秒平均频率，用于排除热降频；
- Git commit、模型权重、相机分辨率与采集格式（MJPEG 1280×720 @30 FPS）保持一致。

#### 4.5.4 测量指标

为充分验证 4.4 节理论分析，本实验在 client 与 server 两端插入逐帧时间戳打点，采集六类指标：

| 类别        | 指标                                                                 | 采集方法                                                                  |
|-----------|--------------------------------------------------------------------|-----------------------------------------------------------------------|
| 吞吐        | <code>fps_mean</code> , <code>fps_p5</code> , <code>fps_p95</code> | 沿 <code>timing_stats.csv</code> 字段                                    |
| 推理耗时      | $T_{\text{infer}}$ 实测分布 (mean, std)                                | <code>policy_server</code> 端 <code>predict_action_chunk()</code> 前后打点 |
| 流水线健康     | stall 帧数比例、stall 平均时长、 <code>must_go</code> 触发次数                   | <code>robot_client</code> 主循环逐帧记录                                     |
| Chunk 利用率 | 每 chunk 过期帧 / 实际执行帧比例                                              | 用 <code>latest_action</code> 与 <code>chunk timestep</code> 计算         |
| 系统资源      | 4 核 CPU 占用率、CPU 温度漂移、CPU 平均频率                                      | <code>psutil</code> + <code>vcgencmd</code> 每秒采样                      |
| 端到端延迟     | 观测采集 $\rightarrow$ 对应 action 执行的时间差                                | <code>TimedObservation</code> / <code>TimedAction</code> 时间戳差         |

其中 **stall 帧比例**与 **chunk 过期帧比例**是支撑 4.4 节不等式分析的关键实证：若理论分析正确，异步模式下应同时观察到显著的 stall 比例（情况 C）和约 60% 的过期帧比例（ $T_{\text{infer}}F/N = 60/100$ ）。

#### 4.5.5 实验流程

**步骤 1：环境准备。**固定 git commit 与模型权重 hash，锁定 CPU governor，关闭 WiFi 等可能引入抖动的后台服务，散热风扇全速。

**步骤 2：模型预热。**两种模式各先跑 5 个 episode 不计入数据，让 PyTorch 算子缓存、相机自动曝光、舵机温度补偿全部进入稳态。

**步骤 3：同步基线采集。**直接运行 `record.py`，沿用第三章 `timing_stats.csv` 的采集脚本，得到 `obs_s`、`inference_s`、`action_s`、`wait_s` 四阶段耗时。

**步骤 4：异步实验采集。**两个终端分别启动：

```
# 终端 1 (policy_server, 绑核 0-2)
taskset -c 0-2 python -m lerobot.scripts.server.policy_server \
--host 127.0.0.1 --port 50051 ...
```

```
# 终端 2 (robot_client, 绑核 3)
taskset -c 3 python -m lerobot.scripts.server.robot_client \
  --server_address 127.0.0.1:50051 \
  --chunk_size_threshold 0.5 ...
```

两端各加日志钩子,按帧打点(`timestep`, `latest_action`, `queue.qsize`, `must_go`, `T_infer`)。

**步骤 5: 数据对齐。**实验后用 **[TODO: ]** 脚本按 `timestep` 把 client 和 server 日志做 join, 得到每帧的 `obs_send_t`、`action_recv_t`、`action_exec_t`, 进而计算端到端延迟与 chunk 过期帧比例。

**步骤 6: 统计分析。**每模式  $\geq 3$  episode 取  $\text{mean} \pm \text{std}$ , 使用配对 t 检验 (按 episode 配对) 评估同步与异步的差异是否显著。

#### 4.5.6 预期结果与待验证假设

基于 4.4 节的理论分析, 本实验预期得到以下四条结果。论文实验数据填入后将逐条核对:

- **H1 (吞吐):**  $|\text{fps\_mean}(\text{异步}) - \text{fps\_mean}(\text{同步})| \leq 10\%$ 。即异步在当前参数下不优于同步, 吻合不等式预测的稳态  $\text{FPS} \approx 20$ 。
- **H2 (stall):** 异步模式下 stall 比例显著大于 0, 且呈现以推理周期为单位的周期性出现, 与图 18 的情况 C 拓扑一致。
- **H3 (chunk 利用率):** 异步模式下每 chunk 约 60% 帧数被丢弃为过期帧, 与理论值  $T_{\text{infer}}F/N = 60/100$  定量吻合。
- **H4 (CPU 争抢):** 异步模式下树莓派 5 实测  $T_{\text{infer}}$  相对 server 独占 4 核时抬高约 10-25% (绑核策略可缓解但不能完全消除)。

H2 和 H3 是不等式分析的直接定量证据。若它们与预测显著不符 (例如 stall 比例  $< 5\%$ 、或过期帧比例  $< 30\%$ ), 则应回到 4.4.4 节重新审视推导前提; 若完全符合预测, 则可作为“异步推理在当前条件下尚不可用”的实证锚点。

#### 4.5.7 实验结果呈现 (待回填)

实验完成后, 本节将以以下五项结果为核心呈现:

1. 图: **FPS 对比柱状图** (同步 vs 异步, 含误差线);

2. 图：稳态时间线甘特图，实测画出执行 / 推理两条 lane，stall 段高亮染色，与图 18 情况 C 作对比；
3. 图：chunk 利用率堆叠柱图，展示每 chunk 中“已过期被丢弃 / 实际执行”的帧数分解；
4. 表：8 项关键指标主结果表（fps、stall%、过期%、CPU%、端到端延迟、 $T_{\text{infer}}$ 、must\_go 触发频次、CPU 温度漂移）；
5. 表：配对 t 检验显著性（同步 vs 异步在 fps 上的  $p$  值）。

[TODO：实验未跑，结果数据与图表待回填；以上图表骨架为最低呈现要求。]

#### 4.5.8 讨论：实验结论的工程含义

预期实验将印证 4.4.5 节的理论预测：在当前  $T_{\text{infer}} = 2 \text{ s}$  下，异步推理的工程收益接近于零。然而这并不意味着异步推理本身没价值——它意味着**异步推理的可用性**强依赖于推理本体的速度。当  $T_{\text{infer}}$  通过量化、ONNX 或 NEON 优化下降到

$$T_{\text{infer}} \leq \frac{N}{2F} = \frac{100}{2 \times 30} \approx 1.67 \text{ s} \quad (5)$$

以下时，不等式重新成立，异步推理才会从“无效”切换为“有效”。也就是说，在本平台上，异步推理是“推理加速完成后”的下游红利，而不是独立可拿的优化。这一判据将直接支撑 4.6 节的优化路线图。

### 4.6 优化路线图与本章小结

#### 4.6.1 优化方向矩阵

综合 4.2–4.5 节的可行性分析与实验预期，将本章涉及的所有候选优化方向汇总如表所示。“可行性”按“结构是否允许”而非“是否值得”判断；“ROI”指当前  $T_{\text{infer}} \approx 2 \text{ s}$  条件下的预期收益。

| 优化方向            | 主要收益      | 可行性 | 复杂度           | 当前 ROI                      |
|-----------------|-----------|-----|---------------|-----------------------------|
| 舵机异步写           | 1–3 ms/帧  | ×   | —             | 负                           |
| 舵机异步读           | 1–2 ms/帧  | ○   | 中（改 SDK）      | 边际                          |
| 推理异步（双进程）       | 消除 CPU 闲置 | ✓   | 高（gRPC + 双进程） | $\approx 0$ （当前 $N$ 不满足不等式） |
| INT8/FP16 量化    | 推理 2–4×   | ✓   | 中             | 高                           |
| ONNX Runtime 后端 | 推理 1.5–2× | ✓   | 低             | 高                           |
| MJPEG → YUYV    | 解码降至 0    | ✓   | 低             | 中                           |
| cs 加大重训         | 拓宽不等式余量   | ✓   | 高（需重训）        | 中                           |

矩阵的核心信息有三条：

- **舵机异步写从结构上排除**，无需进一步评估；
- **推理异步并非独立可拿的优化**，其 ROI 受不等式  $N \geq 2T_{\text{infer}}F$  约束，必须先把  $T_{\text{infer}}$  降下来；
- **真正应当优先的是推理本体加速**（INT8 / ONNX），它既能直接削减主瓶颈，又能间接打开异步推理的可用窗口。

#### 4.6.2 三阶段优化路线

按上述判据，本论文为后续工程演进给出三阶段路线：

**阶段一——推理本体加速**（短期，最高优先级）。目标是把  $T_{\text{infer}}$  从 2 s 降到 1 s 以下。具体路径：ACT 模型 INT8 / FP16 量化、迁移到 ONNX Runtime（其 ARM 后端针对 Cortex-A76 的 NEON 指令做了专门优化）、评估是否需要进一步引入 GEMM 算子手工优化。本阶段单独即可把同步模式的实测 FPS 从 19 推高到 30+。

**阶段二——启用推理异步**（中期，依赖阶段一完成）。当  $T_{\text{infer}} \leq N/(2F) \approx 1.67$  s 时，不等式  $N \geq 2T_{\text{infer}}F$  重新成立，此时启用 `robot_client + policy_server` 双进程架构才能带来真实的吞吐提升。本阶段还需要为 `policy_server` 配置 `taskset` 绑核策略，并验证 4.5 节实验中  $T_{\text{infer}}$  的 CPU 争抢抬升量是否可接受。

**阶段三——可选：模型重训以拓宽不等式余量**（长期）。如果阶段一加速后  $T_{\text{infer}}$  仍接近临界值（例如 1.5 s），可以训练一个 `chunk_size = 200` 的 ACT 模型，让不

等式右侧从  $2 \times 1.5 \times 30 = 90$  远小于  $N = 200$ ，异步推理获得充足余量。但 ACT 的 `chunk_size` 在训练期固定，更大的 open-loop 范围有可能引入误差累积，需配合 temporal ensemble 权重重新调参。本阶段的工作量与不确定性最大，仅在阶段一与阶段二无法单独达到目标 FPS 时才启动。

#### 4.6.3 与第五章的衔接

本章的最终判据可以一句话表述：在树莓派 5 + ACT 的 LeRobot 用例下，推理本体加速是唯一无条件的优化方向，其余优化项的 ROI 都依赖它先完成。第五章“总结与展望”将沿此判据进一步展开，讨论本论文的研究边界、尚未实现的实验项以及面向更高算力嵌入式 NPU 平台（如 RK3588、Jetson Orin Nano）的迁移可能性。

## 第五章 总结与展望

### 5.1 主要工作总结

1. 建立了覆盖 AI 框架到内核的四层性能分析模型；
2. 通过 Python 日志 + ftrace + torch.profiler 多工具联合，端到端量化了各阶段开销；
3. 揭示 ACT 推理（2000–4000 ms）为主瓶颈，系统调用 overhead 可忽略；
4. 推导了异步推理流水线不 stall 的数学条件，指出当前参数配置的局限性；
5. 系统梳理了各优化路径的可行性与成本收益；
6. 对感知、推理、执行三阶段进行了从 Python 到 kernel 的全链路 CallStack 源码分析；
7. 调研了各阶段异步化可行性，论证了舵机异步读/写的不可行性及其原因。

### 5.2 研究局限性

- 仅在 SO101 单臂场景下测试，未覆盖双臂或移动机器人；
- 未实施量化等优化方案并验证，结论停留在分析层面；
- 无 GPU 对比基线，难以量化 CPU 推理与加速器的差距。



### 5.3 未来工作

- INT8 量化 + ONNX Runtime 实验，验证推理加速收益；
- 扩展至多任务场景（更多抓取类别）与多机器人平台；
- 探索面向机器人负载特征的实时操作系统设计。

## 参考文献

- [1] BROHAN A, BROWN N, CARBAJAL J, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control[J]. arXiv preprint arXiv:2307.15818, 2023.
- [2] KIM M J, PUMACAY K, JATAVALLABHULA K M, et al. OpenVLA: An Open-Source Vision-Language-Action Model[J]. arXiv preprint arXiv:2406.09246, 2024.
- [3] CADENE R, ALIBERT S, et al. LeRobot[EB/OL]. 2024. <https://github.com/huggingface/lerobot>.
- [4] ZHAO T Z, KUMAR V, LEVINE S, et al. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware[C]//Proceedings of Robotics: Science and Systems (RSS). 2023.
- [5] CHI C, FENG S, DU Y, et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion[C]//Proceedings of Robotics: Science and Systems (RSS). 2023.
- [6] INTELLIGENCE P.  
pi0: A Generalist Robot Policy[EB/OL]. 2024. <https://www.physicalintelligence.com/>.
- [7] ZHAN T, et al. Mobile ALOHA: Bimanual Mobile Manipulation with Low-Cost Whole-Body Teleoperation[C]//IEEE International Conference on Robotics and Automation (ICRA). 2024.
- [8] ROBOTIS. OpenManipulator[EB/OL]. 2024. [https://emanual.robotis.com/docs/en/platform/openmanipulator\\_x/](https://emanual.robotis.com/docs/en/platform/openmanipulator_x/).
- [9] ROBOTICS O, ROBOTICS C. TurtleBot 4[EB/OL]. 2023. <https://www.turtlebot.com/>.
- [10] HybridRobotics. Berkeley Humanoid Lite[EB/OL]. 2025. <https://lite.berkeley-humanoid.org/>.

## 附 录

## 附录 A：待补充实验清单

| 优先级 | 实验内容                        | 填充章节                       | 预计耗时 |
|-----|-----------------------------|----------------------------|------|
| ★★★ | 推倒/分类任务计时数据                 | 第 3.2.2 节三类任务对比            | 已完成  |
| ★★★ | torch.profiler op-level     | 第 3.2.3 节算子分解              | 1 天  |
| ★★★ | 摄像头层级耗时剖析（实验 06）            | 第 3.1.2 节 Call-Stack 源码分析  | 1 天  |
| ★★  | CPU 频率锁定对比                  | 第 4.2 节峰值成因                | 半天   |
| ★   | perf stat (IPC, cache-miss) | 判断 compute vs memory bound | 半天   |

## 致 谢

[TODO: 致谢内容]